

Yazılım Mimarisi (Software Architecture)

Dr. Öğr. Üyesi Şerif Ali Sadık

Kütahya Dumlupınar Üniversitesi
Mühendislik Fakültesi
Yazılım Mühendisliği Bölümü



2025-2026 Bahar (Spring Semester)



Bölüm 1 - Section 1

Derse Giriş - Course Overview



Dersin İçeriği - Course Overview

- Yazılım Mimarisi ve Mimari Düşünme - Software Architecture and Architectural Thinking
 - Yazılım mimarisinin tanımı ve önemi - Definition and importance of software architecture
 - Mimari kararların etkisi - Impact of architectural decisions
 - Yazılım mühendisliği ile ilişkisi - Relationship with software engineering
- Modülerlik (Modularity)
 - Modülerlik ilkeleri ve avantajları - Principles and benefits of modularity
 - Bağımlılıklar ve gevşek bağlı - Dependencies and loose coupling
 - Uygulama örnekleri - Practical examples of modularity
- Mimari Karakteristikler
 - Mimari kalite öznitelikleri nelerdir?
 - Performans, güvenilirlik, ölçeklenebilirlik gibi temel kavramlar
- Bileşen Bazlı Düşünme (Component-Based Thinking)
 - Bileşen bazlı mimari ve avantajları
 - Bağımlılık yönetimi
 - Uygulamada bileşen bazlı mimari



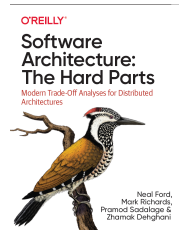
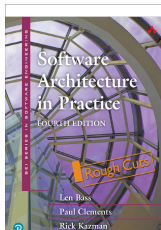
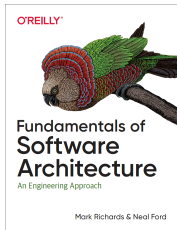
Dersin İçeriği

- Mimari Stillerin Temelleri
 - Yazılım mimari stillerine genel bakış
 - Stillerin karşılaştırılması
- Katmanlı Mimari (Layered Architecture Style)
 - Katmanlı mimarinin prensipleri
 - Katmanlı yapı avantajları ve dezavantajları
- Pipeline (Boru Hattı) Mimari Stili
 - Veri akışı tabanlı mimariler
- Microkernel Mimari Stili
 - Eklenti tabanlı sistemler
- Servis Tabanlı Mimari (Service-Oriented Architecture)
 - SOA, Web servisleri, API yönetimi
- Olay Tabanlı Mimari (Event-Driven Architecture)
 - Publish/Subscribe modeli
- Alan Tabanlı Mimari (Space-Based Architecture)
 - Dağıtık bellek yönetimi ve veri paylaşımı
- Uygun Mimari Stilin Seçilmesi
 - Gerçek dünya senaryoları üzerinden doğru mimari seçimi



Kaynaklar

- Mark Richards, Neal Ford. *Fundamentals of Soft. Architecture: An Engineering Approach*. O'Reilly. 2020
- Len Bass, Paul Clements, Rick Kazman. *Soft. Architecture in Practice*. Pearson, 2nd edition. 2021
- Neal Ford, Mark Richards, *Software Architecture: The Hard Parts*, O'Reilly. 2021.

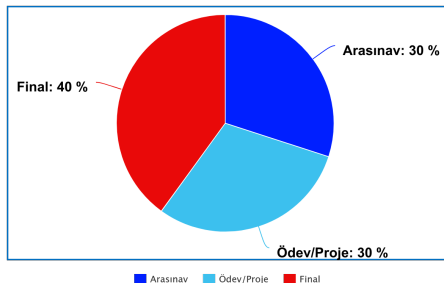


Çevrimiçi Kaynaklar

- DPU Portal
 - *portal.dpu.edu.tr/serifali.sadik*
- Ayrıca derste hafta hafta paylaşılacak ek içerikler



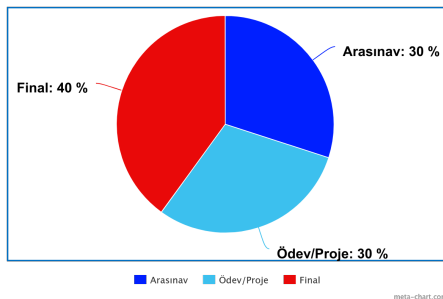
Değerlendirme ve Çıktılar



meta-chart.com



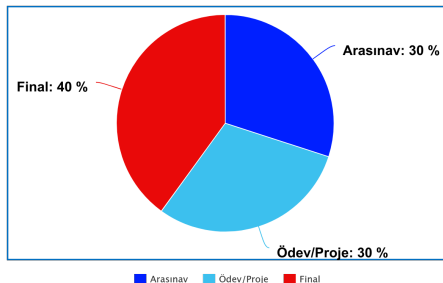
Değerlendirme ve Çıktılar



- Yazılım mimarisi temel kavramlarını ve mimari kalite öz niteliklerini açıklayabilmek
- Farklı yazılım mimari stillerini tanımlar ve uygun kullanım senaryolarını değerlendirebilmek
- Yazılım mimarisini UML, C4 Modeli ve diğer görselleştirme teknikleriyle ifade edebilmek
- Mimari risk analizi yaparak teknik borç, ölçeklenebilirlik ve güvenlik gibi kritik faktörleri değerlendirmek



Değerlendirme ve Çıktılar



meta-chart.com

- Gerçek dünya projelerinde büyük ölçekli sistemleri tasarlayabilme
- Yazılım geliştirme süreçlerinde liderlik yapabilme
- Farklı mimari yaklaşımlar arasında bilinçli seçim yapabilme
- Problemleri çözme



Mimari?

- Architecture: (yun.) ‘chief’ + ‘creator’
- Mimar: (ar.) İmar eden
- Binaların veya **diğer yapıların** planlanması, tasarlanması ve inşa edilmesi süreci



Mimari?

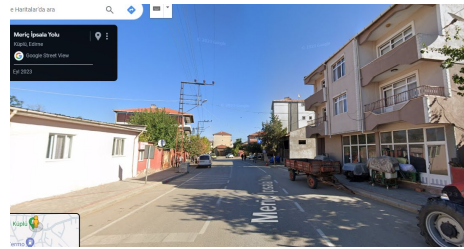
İyi binaların 3 temel direği:

- Utilitas (kullanışlılık):
Kullanışlı olmalı ve onu kullanan insanlar için iyi işlev görmelidir.
- Firmitas (dayanıklılık):
Sağlam bir şekilde ayakta durmak ve iyi durumda kalmak
- Venustas (zarafet/güzellik):
İnsanları memnun etmeli

Yazılım mimarisine de bu üç sac ayağını uygulayabiliriz.



Mimari?



Yazılım Mimarisi

- ISO/IEC/IEEE 42010:2022

Bir yazılım sisteminin mimarisi, arayüzler aracılığıyla etkileşime giren önemli bileşenlerin organizasyonu veya yapısıdır; bu bileşenler birbirini izleyen daha küçük bileşenlerden ve arayüzlerden oluşur.

- Ancak piyasada yazılım mimarı (software architect) kariyeri tam olarak anlaşılmış değil.
- En büyük dezavantaj da teknolojinin ve trendlerin çok hızlı değişmesi.

Who needs an architect? - Martin Fowler

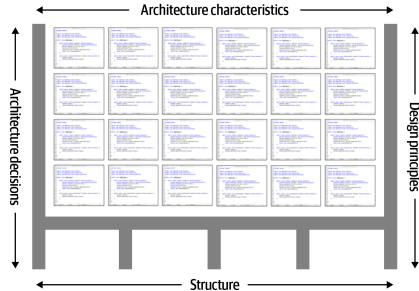
<https://martinfowler.com/ieeeSoftware/whoNeedsArchitect.pdf>



Yazılım Mimarisi

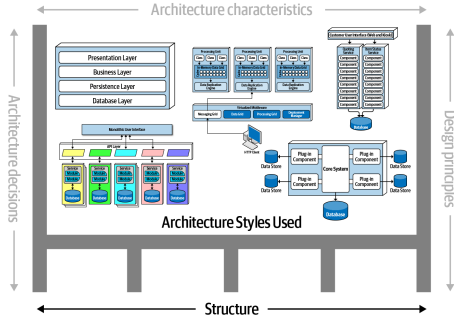
Yazılım mimarisi şu dört yapıtaşından oluşur:

- 1 Yapı (Structure)
- 2 Mimari karakteristikleri
- 3 Mimari kararları (decisions)
- 4 Tasarım ilkeleri



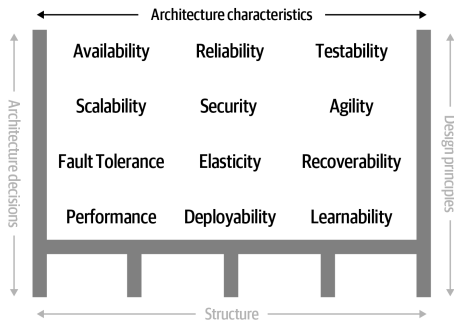
Yazılım Mimarisi - Yapı

- Tercih edilen mimari stilin türünü belirtir: Mikroservis, katmanlı veya mikroçekirdek gibi
- **Yapı, mimarinin kendisini tanımlamaz**
- Uygulamamızın ‘mikroservisler mimarisi’ ile inşa edildiğini söylersek, genel mimariye değil, yapıya atıfta bulunmuş oluruz.



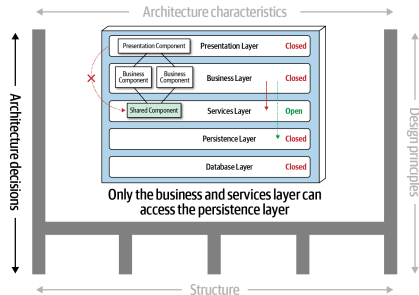
Yazılım Mimarisi - Karakteristikler

- -ebilirlik'ler olarak da tanımlanabilir. Ölçeklenebilirlik, Test edilebilirlik, Güvenilirlik gibi
- Sistemin işlevselliğinden bağımsız olarak sistemin başarı kriterlerinin tanımlanması
- Örneğin, yapıdan bağımsız Test Edilebilirliği desteklemek istiyorsak, yazılımımızı test edilebilir olacak şekilde tasarlamamız gerekir



Yazılım Mimarisi - Kararlar

- Bir sistemin nasıl inşa edilmesi gerektiğine dair kurallardır.
- Örneğin, yapıda hangi katmanların veritabanına erişebileceğini karara bağlamamız gerekebilir.
- Bu kararlar, geliştirici ekipler için kısıtlamalar oluşturur ve neyin yapıp neyin yapılamayacağını belirler.
- Ancak bazı durumlarda, bu kurallar **varyans (variance)** adı verilen istisnalarla aşılabılır.



Yazılım Mimarisi - 2 Kural

İlk Kural

‘Everything in software architecture is a **trade-off**!’

Yazılım mimarisinde her seçim, bir ödün vermeyi gerektirir.

İkinci Kural

‘**Why** is more important than how!’

Neden sorusu nasıl sorusundan daha önemlidir?



Öneriler

Who needs an architect? - Martin Fowler

<https://martinfowler.com/ieeeSoftware/whoNeedsArchitect.pdf>

5 Things Every Developer Should Know About Software Architecture



Sorular.



Bölüm 2 - Section 2
Mimari Düşünme - Architectural
Thinking



Mimari Düşünme Nedir? - What is Architectural Thinking?

- Mimari düşünme, geliştirici (developer) bakış açısından farklıdır. - *Architectural thinking is different from developer thinking.*
- Sadece ‘mimariyi düşünmek’ değil, sistemleri farklı bir perspektiften ele almaktır. - *Not just ‘thinking about architecture’, but approaching systems from a different perspective.*



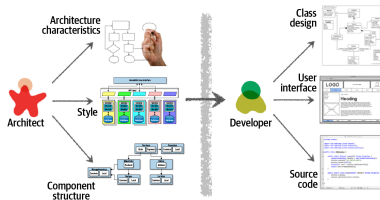
Mimari Düşünmenin Dört Temel Boyutu - Four Dimensions of Architectural Thinking

- Mimari ve tasarım arasındaki farkı anlamak. - **Understanding the difference between architecture and design.**
- Teknik konularda geniş bir bilgiye sahip olmak, ancak belirli bir derinliği de korumak. - **Having broad knowledge of technical topics while maintaining depth in specific areas.**
- Çözümler ve teknolojiler arasındaki ödünleşimleri (trade-off) analiz edip uzlaştırmak. - **Analyzing and reconciling trade-offs between solutions and technologies.**
- İş hedeflerinin önemini anlamak ve bunları mimari kaygılara çevirebilmek. - **Understanding the importance of business goals and translating them into architectural concerns.**



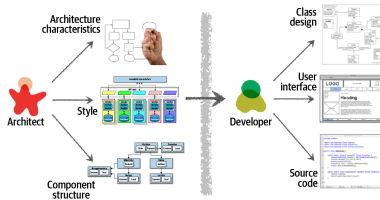
Mimar Gibi Düşünmek - Thinking Like an Architect

- Mimari düşünmek, mimari ve tasarım arasındaki farkı bilmektir. - **Architectural thinking is knowing the difference between architecture and design.**
 - İş gereksinimlerini analiz edip bunları mimari özelliklere dönüştürmek. - **Analyzing business requirements and translating them into architectural features.**
 - Kullanılacak mimari desenleri ve stilleri belirlemek. - **Determining architectural patterns and styles to be used.**
 - Sistemin yapı taşlarını (bileşenleri) oluşturmak. - **Creating the building blocks (components) of the system.**



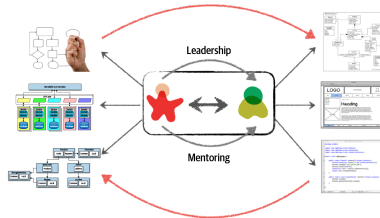
Geleneksel Modelin Sorunu

- Geleneksel modelde mimar, geliştirme ekibinden ayrıdır. - **In the traditional model, the architect is separate from the development team.**
- Mimari kararlar bazen geliştirme ekibine ulaşmaz. - **Architectural decisions sometimes do not reach the development team.**
- Geliştirme ekibinin aldığı kararlar da mimara geri dönmez. - **Decisions made by the development team do not feed back to the architect.**



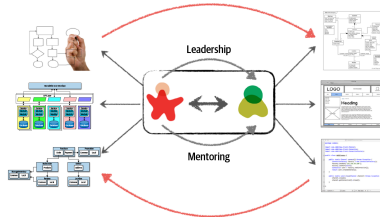
Mimar ve Geliştirici Aynı Takımda Olmalı

- Mimariyi etkin hale getirmek için engelleri kaldırmalıyız. - **To make architecture effective, we need to remove barriers.**
- Mimar ve geliştiriciler aynı takımda çalışmalıdır. - **Architects and developers should work in the same team.**
- Bu yaklaşım, mimarın geliştiricilere rehberlik etmesini sağlar. - **This approach allows the architect to guide the developers.**



Mimari ve Tasarımın Sınırı

- Mimari nerede biter, tasarım nerede başlar? - **Where does architecture end and design begin?**
- Aslında birbirlerinden bağımsız değildirler. - **They are not actually independent of each other.**
- Yazılım projesi yaşam döngüsü boyunca paralel şekilde var olurlar ve senkronize tutulmaları gerekir. - **They exist in parallel throughout the software project lifecycle and need to be kept in sync.**



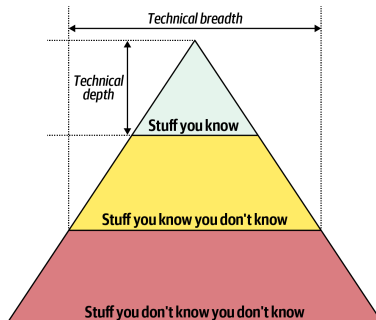
Teknik Derinlik - Technical Depth

- Geliştiriciler, belirli teknolojilerde **derin** uzmanlık kazanmayı hedefler. - *Developers aim to gain deep expertise in specific technologies.*
- Bir dili, framework'ü veya aracı en ince ayrıntısına kadar öğrenir ve ustalaşır. - *They learn and master a language, framework, or tool in great detail.*
- Mimarlar ise farklı teknolojiler hakkında **geniş** bilgi sahibi olmalıdır. - *Architects, on the other hand, should have broad knowledge of different technologies.*
- Birçok teknolojiye yüzeysel ama yeterince bilinçli şekilde hakim olmalı ve doğru aracı doğru yerde nasıl kullanacağını bilmelidir. - *They should be superficially but sufficiently knowledgeable about many technologies and know how to use the right tool in the right place.*



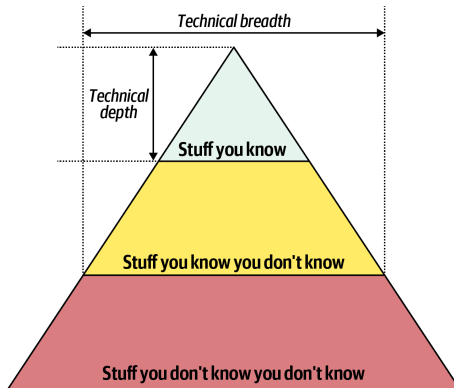
Bilgi Piramidi - Knowledge Pyramid

- Mimarın değeri, teknolojileri geniş bir perspektiften ele alarak doğru seçimleri yapabilmesi ve çözümleri yüksek seviyede tasarlayabilmesi ile ölçülür. - *The value of an architect is measured by their ability to take a broad perspective on technologies, make informed choices, and design solutions at a high level.*
- Burada bilgi piramidi devreye giriyor: - *This is where the knowledge pyramid comes into play:*



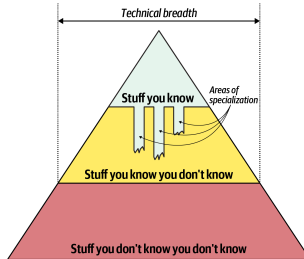
Bilgi Piramidi

- Bir yazılım mimarı için önemli olan üst ve orta katmanlara hakim olmaktır.
- Ne kadar çok bilinmeyeni keşfeder ve yeni teknolojilere açık olursa, teknik genişliği o kadar artar.



Bilgi Piramidi

- Bir geliştiricinin mimar rolüne geçmesi zor olabilir.
- Bu süreçte iki yaygın hata yapılır:
 - Her konuda uzman olmaya çalışmak ve hiçbirinde derinleşememek
 - Mevcut bilgiyi güncellemek ve eski teknolojileri hâlâ güncel sanmak
- Bu yüzden bir mimarın, bazı uzmanlıklarını bilinçli olarak bırakıp daha geniş bir teknoloji portföyü oluşturması gerekir.
- Mimari düşünme biçimi, uzmanlaşmak değil, farklı uzmanlıkları anlamak ve bir araya getirmek üzerine kuruludur.



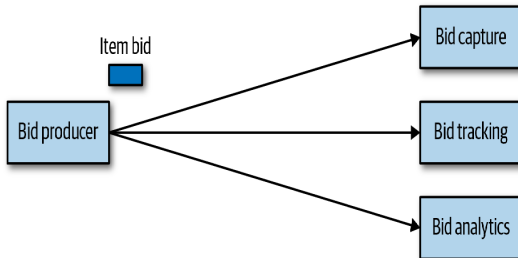
Ödünleri (Trade-offs) Analiz Etme

- Mimarlık tamamen trade-off'larla ilgilidir:
 - Mimarlık, Google'da bulamayacağınız şeydir.
 - Mimarlıkta doğru ya da yanlış cevaplar yoktur, yalnızca trade-off'lar vardır.
 - Her şeyin cevabı: 'Duruma bağlıdır' (It depends.)

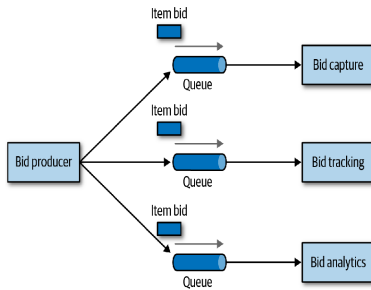
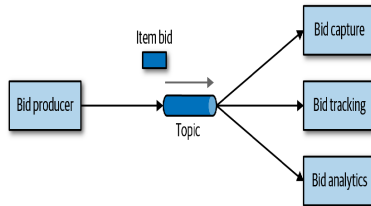


Bir örnek

- Trade-off'ları görmek ve analiz etmek mimari düşünmenin temelidir.
- Örnek: Bir açık artırma sisteminde tekliflerin işlenmesi
- İki alternatif çözüm var:
 - Topic (Yayın-Abone, Pub-Sub)
 - Queue (Noktadan Noktaya, Point-to-Point)



Bir örnek



Örnek - Topic

- Bir üretici (publisher) mesajı bir konuya (topic) gönderir ve birden fazla tüketici (subscriber) bu mesajı alabilir.
 - Üretici (Publisher): Mesajı bir "topic"e gönderir.
 - Tüketiciler (Subscribers): Belirli bir "topic"e abone olan tüm tüketiciler bu mesajı alır.
 - Yeni bir tüketici eklemek için sadece o topic'e abone olması yeterlidir.
- Bir e-ticaret sisteminde sipariş onayı bildirimi gönderen bir servis düşünelim.
 - Sipariş oluşturulduğunda Sipariş Servisi bir "Sipariş Onaylandı" mesajı yayınlar.
 - Fatura Servisi, Müşteri Bildirim Servisi ve Stok Yönetim Servisi bu konuya abonedir.
 - Mesajı aldıklarında her biri kendi görevlerini yerine getirir: faturayı oluşturur, müşteriye e-posta gönderir, stokları günceller.



Örnek - Topic

● Avantajları:

- ✓ Genişletilebilirlik: Yeni tüketiciler eklemek kolaydır.
- ✓ Bağımsızlık (Loosely Coupled): Üretici, tüketicileri tanımaz; sadece bir "topic"e mesaj gönderir.
- ✓ Aynı mesaj birden fazla servis tarafından tüketilebilir.

● Dezavantajları:

- ✗ Güvenlik ve erişim kontrolü zor olabilir: Herkes abone olabilir ve mesajları okuyabilir.
- ✗ Takip etmesi zor olabilir: Hangi servislerin mesajları aldığını ve işlediğini izlemek için ek mekanizmalara ihtiyaç vardır.
- ✗ Heterojen veri formatı desteklemez: Tüm aboneler aynı veri formatını kullanmak zorundadır.



Örnek - Queue

- Bir üretici (producer) mesajı bir kuyruk (queue) içine bırakır ve yalnızca bir tüketici (consumer) mesajı alır ve işler.
 - Üretici (Producer): Mesajı bir kuyruk içine koyar.
 - Tüketici (Consumer): Kuyruğa bağlanarak mesajı çeker ve işler.
 - Mesaj tüketildikten sonra kuyruktan silinir.
- Bir banka sistemi düşünelim. Bir ATM'den para çekildiğinde, işlem kuyruğa eklenir:
 - "Para Çekme İşlemi" mesajı kuyruğa konur.
 - Banka İşlem Servisi bu mesajı kuyruğundan çeker ve hesabı günceller.
 - Mesaj işlendiği için kuyruğa konulan mesaj başka bir servis tarafından tekrar işlenmez.



Örnek - Queue

● Avantajları:

- ✓ Güvenli mesaj işleme: Bir mesaj sadece bir tüketici tarafından işlenir.
- ✓ Özel mesajlaşma sağlar: Her tüketici yalnızca kendisine özel kuyruğa gelen mesajları alır.
- ✓ Her tüketici, kendine uygun mesaj formatını belirleyebilir.

● Dezavantajları:

- ✗ Sıkı bağıklık (Tightly Coupled): Üretici, hangi tüketiciye mesaj gönderdiğini bilmek zorundadır.
- ✗ Yeni tüketici eklemek için ekstra yapılandırma gerekir.
- ✗ Paralel işlemeyi zorlaştırabilir: Aynı mesaj birden fazla tüketici tarafından işlenemez.



Topic vs. Queue Karşılaştırması

Kriter	Topic (Pub/Sub)	Queue (Point-to-Point)
Mesaj kaç tüketici alır?	Birden çok tüketici alabilir	Sadece bir tüketici alır
Yeni tüketici ekleme	Kolay, sadece abone olmak yeterli	Ekstra yapılandırma gerektirir
Bağlılık (Coupling)	Gevşek bağlı (Loosely Coupled)	Sıkı bağlı (Tightly Coupled)
Kullanım senaryosu	Bildirimler, olay bazlı sistemler	İşlemler, görev kuyrukları
Mesaj güvenliği	Daha az güvenli (Herkes abone olabilir)	Daha güvenli (Sadece belirli tüketiciler alabilir)
Ölçeklenebilirlik	Daha kolay ölçeklenebilir	Daha kontrollü ölçeklenebilir
Veri formatı desteği	Tüm tüketiciler aynı veri formatını kullanır	Her tüketici kendi formatını belirleyebilir



Sorular.



Bölüm 3

Modülerlik



Modülerlik - Modularity

“Yazılım mimarisi hakkındaki sözlerin %95’i ‘**modülerliğin**’ faydalarını övmek için harcanır ve bunun nasıl başarılacağı hakkında çok az şey söylenir.”

— Glenford J. Myers (1978)



Modülerlik

- **Modülerlik**, yazılımı düzenlemek için kullanılan bir prensiptir.
- İlgili kodları bir araya getirerek sistemin düzenli olmasını sağlar.
- Yazılım sistemleri zamanla **düzensizliğe (entropy)** doğru ilerler.
- Bu yüzden mimarların özenle düzeni koruması gerekir.
- **Modülerlik**, kendi başına bir **mimari karakteristik** değildir.
- Projeler için doğrudan bir gereksinim olarak tanımlanmaz.
- Ancak her yazılım projesinin **düzeni ve tutarlılığı** sağlaması zorunludur.



Modülerlik

- Geliştirme sırasında (**development time**) projenin **modüllere** ayrıştırılmasıdır.
- Modüller bağımsız olarak geliştirilebilir.



Modül

- **Modül**, ‘daha karmaşık bir yapı oluşturmak için kullanılabilen bir dizi standartlaştırılmış parça veya bağımsız birim’ olarak tanımlanır.
- Birbiriyle ilişkili kodların mantıksal olarak gruplandırılmasını modülerlik olarak tanımlarız.
- Modülerlikten kodun gruplandırılması olarak bahsettiğimizde, fiziksel bir ayırmadan değil, mantıksal bir ayırmadan bahsediyoruz.



Modüller (Modules)

- Modüller, kaynak kod seviyesinde organizasyon birimleridir.
- Genellikle bir grup ilgili sınıf, fonksiyon, veri yapısı veya başka modülleri içeren mantıksal yapılardır.
- Geliştirme zamanında (*development-time*) var olurlar ve bağımlılıkları belirlerler.
- Örnekler:
 - Java'da **package**
 - Python'da **module** (.py dosyaları)
 - C#'ta **namespace**



Bileşenler (Components)

- Bileşenler, çalışma zamanı (*run-time*) sırasında etkileşime giren bağımsız ve dağıtılabılır birimlerdir.
- Bir sistemin çalışırken birbirleriyle iletişim kurabilen, genellikle bir arayüz sağlayan bölümleridir.
- Derlenmiş (*compiled*) ve deploy edilmiş yazılım birimleridir.
- Örnekler:
 - Java'da bir **Spring Bean**
 - .NET'te bir **Service** veya **DLL**
 - Mikroservis mimarisinde bir **bağımsız servis**



Örnek (Modül vs. Bileşen)

Bir e-ticaret uygulaması düşündüğümüzde:

- **Geliştirme zamanında (development time)**

- `PaymentModule`: Ödeme ile ilgili tüm fonksiyonları içeren bir modüldür.
- `UserModule`: Kullanıcı işlemleriyle ilgili kodları barındırır.

- **Çalışma zamanında (run-time)**

- `PaymentService`: Gerçek çalışan ödeme servisi bir bileşendir (API olarak çalışır).
- `UserService`: Kullanıcı verilerini yöneten bileşendir (başka sistemlerle iletişime geçer).



Örnek (Modül vs. Bileşen)

- Zaman dilimi:
 - Modül(Module): Geliştirme zamanında (development time) var olur.
 - Bileşen(Component): Çalışma zamanında (run-time) var olur.
- Temel Amacı:
 - Modül(Module): Kodun mantıksal organizasyonu ve geliştirme sürecinde bağımsız geliştirme birimleri oluşturmak.
 - Bileşen(Component): Çalışma zamanında birbirleriyle etkileşime girebilen bağımsız ve dağıtılabilir birimler oluşturmak.
- Fiziksel Karşılık:
 - Modül(Module): Genellikle kaynak kod seviyesinde organizasyon birimleri (örneğin, Java package (.java), Python module (.py)).
 - Bileşen(Component): Derlenmiş ve deploy edilmiş yazılım birimleri (örneğin, mikroservis, DLL).



Big Ball of Mud

Big Ball of Mud - Foote & Yoder, 1997

<https://hillside.net/plop/plop97/Proceedings/foote.pdf>

- Big Ball of Mud, düzensiz, iyi tanımlanmamış, kötü organize edilmiş ve sürdürülebilirliği zor bir anti-desendir(anti-pattern).
- Genellikle plansız gelişim, aceleyle yapılan değişiklikler nedeniyle oluşur.
- Başlangıçta küçük ve yönetilebilir olabilirler, ancak zamanla kodun karmaşıklığı arttıkça bakım yapmak zorlaşır.



Big Ball of Mud

Big Ball of Mud belirtileri:

- **Düzensiz ve parçalanmış kod:** Kodun belirli bir mimari veya organizasyon prensibine uymaması.
- **Bağımlılık karmaşası:** Her şeyin her şeyle bağlantılı olması, bir değişikliğin tüm sistemi etkilemesi.
- **Anlaşılması zor ve dökümantasyonsuz kod:** Yeni geliştiricilerin projeyi anlamakta zorlanması.
- **Geçici çözümlerle dolu sistem:** "Şimdilik böyle çalışsın" diyerek eklenen geçici yamaların kalıcı hale gelmesi.



Big Ball of Mud

Big Ball of Mud ne zaman karşımıza çıkar?

- Acil ve hızlı çözüm arandığında.
 - Mimari tasarlamaya vakit yoktur.
 - Tasarımsız işe girişmek gerekebilir (No design up front).
 - Anlık sorunlara hızlı çözümler üretilirken ‘topumuz’ kartopu gibi büyür.
- Maliyetsiz bir çözüm arandığında.
 - Projede asıl kaygı maliyet olduğunda çözüm olarak düşünülebilir.
 - Proje ilerlede genişlerse ters tepebilir.



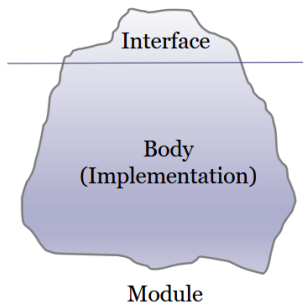
Problemler:

- Yüksek bakım maliyetleri
- Belirli bir noktada düşük esneklik
 - Başlangıçta çok esnek olabilir.
 - Bir süre sonra güncellemeler kabusa dönüşebilir.
- Atalet
 - Sistem Büyük Bir Çamur Topu haline geldiğinde onu dönüştürmek çok zordur.
 - Ya ilk geliştiriciler ya da çok uzman geliştiriciler nereye dokunacağını bilebilir.
 - Temiz kod geliştiricileri bu sistemlerden kaçar.



Modüller

- Modül, belirli bir **sorumluluk kümesini** yerine getiren yazılım birimidir.
- Geliştirme aşamasında (**build-time**) anlamlıdır, çalıştırma anında (run-time) değil.
- Arayüzü (interface) ve gövdeyi (body/implementation) ayırır.



Buzdağı Metaforu ve Modüller

● Buzdağının suyun üstündeki kısmı (Interface)

- Modülün tanımını yapar ("*bu modül ne işe yarıyor?*").
- Kullanıcıların nasıl etkileşime gireceğini belirler (kullanım sözleşmesi - *contract*).
- Dış dünyaya açık olan fonksiyonlar, sınıflar, API'ler buradadır.

● Buzdağının suyun altındaki kısmı (Body/Implementation)

- Modülün gerçek işleyişini içerir.
- Kullanıcılar doğrudan görmez ve erişemez.
- İçerisinde fonksiyonların detaylı implementasyonu, veri yapıları ve algoritmalar bulunur.

● Buzdağının tamamı (Module)

- Bir modül, hem arayüzü hem de gövdeyi kapsar.
- Kullanıcılar sadece interface ile etkileşime girer, ancak asıl iş body kısmında yapılır.



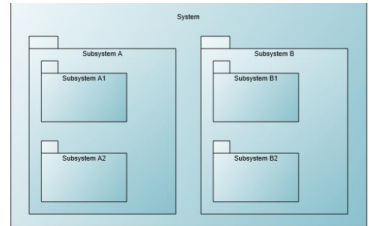
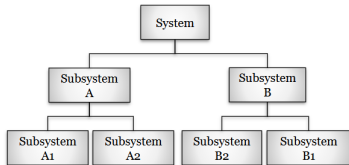
Arayüz Tasarımı için Örnek

```
interface Payment {  
    void processPayment();  
}  
  
class CreditCardPayment implements Payment {  
    public void processPayment() { /* Kredi kartı işlemleri */ }  
}  
  
class PayPalPayment implements Payment {  
    public void processPayment() { /* PayPal işlemleri */ }  
}  
  
class PaymentProcessor {  
    void process(Payment payment) {  
        payment.processPayment();  
    }  
}
```



Modüllere Ayırıştırma

- Birbirleriyle ilişkileri: Bir grubun/kümenin parçası olmak şeklindedir.
- Kısıtlamalarımız:
 - Döngülere izin yok.
 - Genellikle bir modülün bir ebeveyni olur.



Modülerliğin Kalite Nitelikleri

- **Sürdürülebilirlik**

- Değişiklikleri ve genişletmeleri kolaylaştırır.
- Lokal işlevsellik

- **Basitlik**

- Bir modül yalnızca bir arayüz sunar. - daha az karmaşıklık

- **Yeniden Kullanılabilirlik**

- Modüller başka bağlamlarda da kullanılabilir.

- **Bağımsızlık**

- Modüller farklı ekipler tarafından geliştirilebilir.



Modülerliğin Zorlukları(Challenges)

- Vasat/yanlış ayrıştırma karmaşıklığı artırabilir.
- Bağımlılık yönetimi zorluk çıkarabilir.
- Modüllerin ayrıştırılması ekip organizasyonunu etkiler.
- Geliştirme ile satın alma arasındaki bir trade-off karşımıza çıkacaktır (COTS vs. FOSS).



Modüler Tasarım Önerileri

- SOLID tasarım ilkeleri (Robert C. Martin)
 - SRP (Single Responsibility Principle)
 - OCP (Open-Closed Principle)
 - LSP (Liskov Substitution Principle)
 - ISP (Interface Segregation Principle)
 - DIP (Dependency Inversion Principle)
- Cohesion (Bağlılık/Tutarlılık)
- Coupling (Bağımlılık)
- Connascence (Bağımlı Evrimleşme)
- Robustness: Postel's Law (Sağlamlık: Postel Yasası)
- Demeter's Law (Demeter Yasası)
- Fluid Interfaces (Akışkan Arayüzler)



Tek Sorumluluk İlkesi

- “Her sınıfın (veya modülün) yalnızca tek bir sorumluluğu olmalıdır.”
- Bir sınıfın sadece bir iş yapması ve yalnızca bir değişiklik nedeni olması gerekir.
- Eğer bir sınıfın birden fazla sorumluluğu varsa, bu sınıfın değişmesi gerekecek birçok sebep olabilir. Bu da bakımı zorlaştırır.



Kötü Örnek

```
class ReportManager {  
    void generateReport() { /* Rapor oluşturma */ }  
    void saveReportToFile() { /* Dosyaya kaydetme işlemi */ }  
    void printReport() { /* Yazdırma işlemi */ }  
}
```

Tek Sorumluluğa Örnek

```
class ReportGenerator {  
    void generateReport() { /* Rapor oluşturma */ }  
}  
  
class ReportSaver {  
    void saveReportToFile() { /* Dosyaya kaydetme işlemi */ }  
}  
  
class ReportPrinter {  
    void printReport() { /* Yazdırma işlemi */ }  
}
```

Açık/Kapalı İlkesi

- "Bir modül, genişlemeye açık ancak değişime kapalı olmalıdır."
- Yeni bir özellik eklemek için mevcut kodu değiştirmek yerine genişletmeliyiz.
- Değiştirme gerektirmeyen, genişletilebilir yapılar oluşturmak önemlidir.

Kötü Örnek

```
class PaymentProcessor {  
    void processPayment(String type) {  
        if (type.equals("CreditCard")) {  
            // Kredi kartı işlemleri  
        } else if (type.equals("PayPal")) {  
            // PayPal işlemleri  
        }  
    }  
}
```



Açık/Kapalı İlkesine Örnek

```
interface Payment {  
    void processPayment();  
}  
  
class CreditCardPayment implements Payment {  
    public void processPayment() { /* Kredi kartı işlemleri */ }  
}  
  
class PayPalPayment implements Payment {  
    public void processPayment() { /* PayPal işlemleri */ }  
}  
  
class PaymentProcessor {  
    void process(Payment payment) {  
        payment.processPayment();  
    }  
}
```



Liskov Yerine Koyma İlkesi

- “Bir alt sınıf (subclass), üst sınıfın (superclass) yerine sorunsuzca geçebilmelidir.”
- Eğer bir sınıfın yerine türetilmiş bir sınıfı koyduğumuzda sistem hatasız çalışıyorsa, LSP sağlanmıştır.
- Türev sınıflar, üst sınıfın davranışını bozmamalıdır.
- Yaygın hatalar:
 - Temel sınıfın davranışını devralma ve değiştirme
 - Üst sınıfa, alt sınıfın takip etmediği işlevsellik eklemek



Arayüz Ayrımı İlkesi

- İstemciler kullanılmayan metotlara bağımlı olmamalıdır.
- Küçük ve uyumlu arayüzlere sahip olmak daha iyidir.
- Aksi takdirde istenilmeyen bağımlılıklar ortaya çıkacaktır.
- Bir modül kullanılmayan işlevlere bağlıysa ve bu işlevler değişirse etkilenebilir.



Bağımlılıkların Ters Çevrilmesi İlkesi

- **Üst seviye modüller, alt seviye modüllere bağımlı olmamalıdır.**
- Her ikisi de soyutlamalara (interfaces/abstractions) bağımlı olmalıdır.
 - Katı bağımlılıkları azaltmak için üst seviye bileşenlerin doğrudan alt seviye bileşenlere bağımlı olması engellenir.

Kötü Örnek

```
class MySQLDatabase {  
    void connect() { /* MySQL bağlantısı */ }  
}  
  
class DataManager {  
    MySQLDatabase db = new MySQLDatabase();  
    void fetchData() { db.connect(); }  
}
```

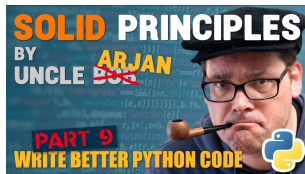


Öneriler

SOLID ilkeleri için kaynak kodlar

<https://github.com/ArjanCodes/betterpython/tree/main/9-solid>

SOLID ilkeleri için video kaynakları



Bağlılık - Cohesion

- Bir modül içindeki parçaların birbirleriyle olan ilişkisini ifade eder.
- Yüksek bağlılık, bir modüldeki öğelerin mantıksal ve işlevsel olarak birbirine bağlı olması anlamına gelir.
- Düşük bağlılık, bir modülde ilgisiz veya zayıf bağlı öğelerin bulunmasıdır.



Bağlılık Tipleri

- **Functional Cohesion (Fonksiyonel bağlılık):** Modüldeki tüm öğeler tek bir işlevi gerçekleştirmek için bir araya getirilmiştir. Bu en iyi cohesion türüdür.
- **Sequential Cohesion (Sıralı bağlılık):** Bir modülün çıktısı, başka bir modülün girdisi olur.
- **Communicational Cohesion (İletişimsel bağlılık):** İki modül aynı veri üzerinde çalışıyorsa veya ortak bir çıktıya katkıda bulunuyorsa. (Örn: Veritabanına veri yazıp, aynı veriyle rapor oluşturmak)
- **Procedural Cohesion (Prosedürel bağlılık):** Modüller belirli bir sırayla çalıştırılmalıdır.
- **Temporal Cohesion (Zamansal bağlılık):** Aynı anda çalıştırılması gereken işlemler aynı modülde yer alır (Örn: Bilgisayar açılırken yapılan işlemler).
- **Logical Cohesion (Mantıksal bağlılık):** Modüldeki öğeler mantıksal olarak birbirine benzer ama işlevsel olarak farklıdır (Örn: Java'daki StringUtils sınıfı).
- **Coincidental Cohesion (Tesadüfi bağlılık):** Modülde bulunan öğeler birbiriyle tamamen ilgisizdir ve yalnızca aynı dosyada yer alır. Bu en kötü cohesion türüdür.



Lack of Cohesion of Methods (LCOM)

LCOM metriği, bir sınıftaki metotların birbirleriyle ve sınıfın değişkenleriyle ne kadar ilişkili olduğunu ölçer.

LCOM Formülü:

$$LCOM = \begin{cases} P - Q, & \text{eğer } P > Q \\ 0, & \text{diğer} \end{cases}$$

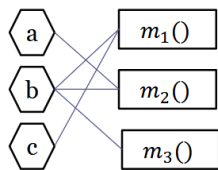
- P : Birbirinden bağımsız metot çiftlerinin sayısı
- Q : Ortak bir alanı (değişken) paylaşan metot çiftlerinin sayısı

Yorum:

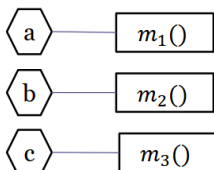
- **LCOM= 0** → Sınıfın bağlı olduğu anlamına gelir (İyi tasarım).
- **LCOM > 0** → Sınıfta bağılılık düşüktür (Kötü tasarım).



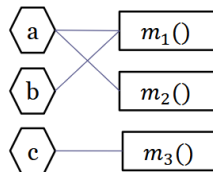
Lack of Cohesion of Methods (LCOM)



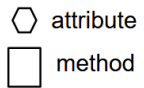
$|P| = 0, |Q| = 3$
LCOM=0



$|P| = 3, |Q| = 0$
LCOM=3



$|P| = 1, |Q| = 2$
LCOM=0



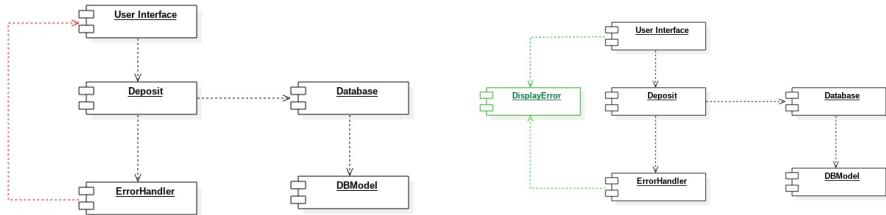
Bağımlılık - Coupling

- Bir modülün/sınıfın başka bir modüle/sınıfa ne kadar bağımlı olduğunu gösterir.
- Düşük bağımlılık (loose coupling), modüllerin birbirine bağımlılığının az olmasıdır. Esneklik, ölçeklenebilirlik ve bakım kolaylığı sağlar.
- Yüksek bağımlılık (tight coupling), modüllerin birbirine sıkı sıkıya bağımlı olmasıdır. Değişiklik yapmak zorlaşır çünkü bir modülde yapılan değişiklik diğerlerini etkileyebilir.



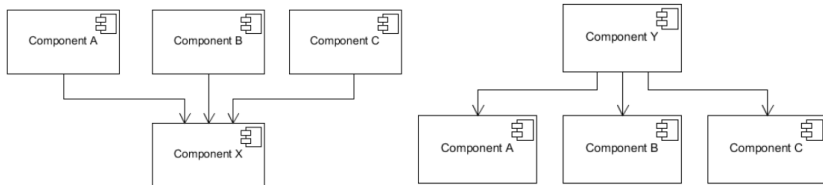
Bağımlılık İlkeleri-1 (Döngüsel bağımlılıklar ilkesi)

- Döngülerden kaçının.
- Bir döngü, tek bir değişikliği çok zor hale getirebilir.
- Çok sayıda modül etkilenir.



Bağımlılık İlkeleri-2 (Kararlı bağımlılıklar ilkesi)

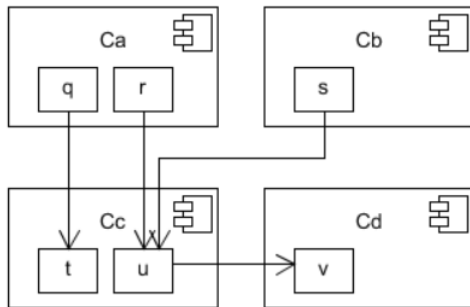
- Bir tasarımdaki bileşenler arasındaki bağımlılıklar kararlılık yönünde olmalıdır
- Bir bileşen yalnızca kendisinden daha kararlı olan bileşenlere bağımlı olmalıdır
- Kararlılık= değiştirmek için daha az neden



Bağımlılık İlkeleri-2 (Kararlı bağımlılıklar ilkesi)

$$I = \frac{Fan_{out}}{Fan_{in} + Fan_{out}}$$

- $I \rightarrow$ kararsızlık (instability)
- $Fan_{in} \rightarrow$ Gelen (giren) bağımlılıklar
- $Fan_{out} \rightarrow$ Giden (çıkan) bağımlılıklar



$$I(Ca) = \frac{2}{0+2} = 1$$

$$I(Cb) = \frac{1}{0+1} = 1$$

$$I(Cc) = \frac{1}{3+1} = \frac{1}{4}$$

$$I(Cd) = \frac{0}{1+0} = 0$$



Soyutluk(Abstractness)

- Abstractness, soyut öğelerin (abstract class, interface vb.) somut öğelere (implementations) oranıdır.
- Ancak, çok fazla soyutlama da kodun nasıl bir araya geldiğini anlamayı zorlaştırabilir.

$$A = \frac{\sum m_a}{\sum m_c}$$

Örnek: Eğer bir uygulama 5000 satırlık kodu sadece `main()` içinde barındırıyorsa:

- Soyut öğe sayısı= 1
- Somut öğe sayısı= 5000

Bu durumda soyutluk neredeyse 0'dır.



Bağımlılık İlkeleri-3 (Kararlı Soyutlamalar İlkesi)

- Bir modül kararlılık ve soyutluluk dengesinde olmalıdır.
- Maksimum düzeyde kararlı olan paketler maksimum düzeyde soyut olmalıdır.
- Kararsız paketler somut olmalıdır.
- Soyutluk ile kararsızlık arasındaki ideal ilişki:

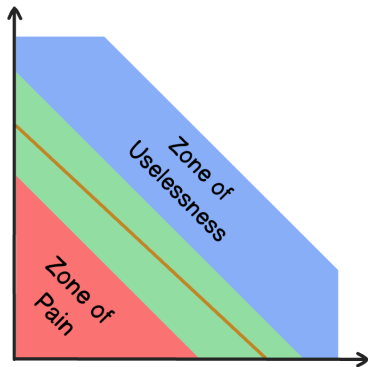
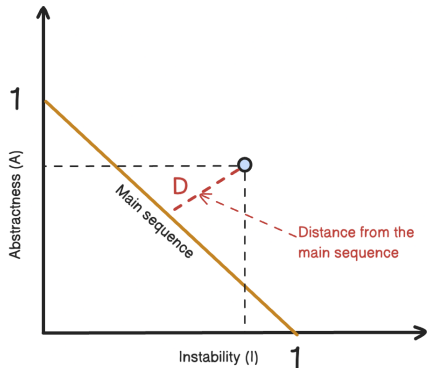
$$D = |A + I - 1|$$

- **D** → Ana Çizgiden Uzaklık (Distance from the Main Sequence)
- **D= 0** → İdeal denge noktası
- **D > 0** → Modül, ya çok soyut ve stabil ya da çok somut ve kararsızdır

Amaç: D değerini mümkün olduğunca sıfıra yaklaştırmaktır.



Bağımlılık İlkeleri-3 (Kararlı Soyutlamalar İlkesi)



Connascence Nedir?

- Connascence, iki veya daha fazla yazılım bileşeni arasındaki ilişki türünü ve gücünü ifade eder.
- Bileşenler arasındaki değişimlerin sistemde nasıl yayılacağını anlamamıza yardımcı olur.

Connascence Türleri

- Statik (derleme zamanında tespit edilebilen)
- Dinamik (çalışma zamanında ortaya çıkan)



Statik Connascence Türleri

- **Connascence of Name:** İsimlerin tutarlı kullanımı gereklidir.
- **Connascence of Type:** Veri türleri arasındaki ilişkiyi gösterir.
- **Connascence of Meaning:** Değişkenlerin belirli bir anlama sahip olması gerekir.
- **Connascence of Position:** Parametrelerin belirli bir sırayla kullanılması gereklidir.
- **Connascence of Algorithm:** İki bileşenin aynı algoritmayı takip etmesi gerekir.

```
public class Time {  
    int _hour; int _min; int _sec;  
  
    public Time(int hour, int min, int sec) {  
        _hour = hour ;  
        _minute = minute ;  
        _second = second ;  
    }  
  
    public String display() {  
        return _hour + ":" + _min + ":" + _sec ;  
    }  
}  
  
public class Client {  
    val noon = Time(12,0,0);  
    . . .  
}
```



Dinamik Connascence Türleri

- **Connascence of Execution:** Belirli bir çalışma sırası takip edilmelidir.
- **Connascence of Timing:** İşlemlerin belirli zamanlamalarla çalışması gerekir.
- **Connascence of Value:** Belirli değerlerin uyum içinde olması gereklidir.
- **Connascence of Identity:** Kimliklerin sistem içinde tutarlı olması gerekir.

```
Email email = new Email();  
email.setRecipient("foo@example.comp");  
email.setSender("me@mydomain.com");  
email.send();  
email.setSubject("Hello World");
```



Postel Yasası (Robustness Principle)

Tanım: Postel'in Yasası (1980), TCP/IP için tanımlanmıştır:

*"Be liberal in what you accept and conservative in what you send."
(Gelen veride esnek ol, gönderdiğin veride katı ol.)*

Faydaları:

- Farklı sistemler arasında **uyumluluğu artırır**.
- **İyi biçimlendirilmiş mesajlar** göndererek sistemlerin sorunsuz çalışmasını sağlar.
- Hatalı mesajları **mümkün olduğunca kabul ederek** sistem esnekliğini artırır.



Demeter's Law (Principle of Least Knowledge)

Tanım:

- **Az bilgi prensibi** olarak da bilinir.

İlkeler:

- Bir birim, **yalnızca doğrudan ilişkili olduğu birimler hakkında bilgi sahibi olmalıdır.**
- Yalnızca **"arkadaşlarıyla"** iletişim kurmalı, **"yabancılarla"** konuşmamalıdır.

Kötü Tasarım Belirtileri:

- Nesne zincirleme erişimleri (**çok fazla nokta kullanımı**):
a.b.method(...) yerine
a.method(...) tercih edilmelidir.



Sorular.



Bölüm 4

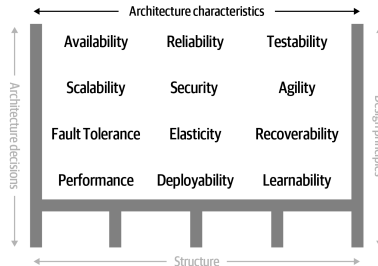
Mimari karakteristikler



Mimari Karakteristikler

Architectural characteristics (mimari karakteristikler);

- Bir yazılım sisteminin sahip olması gereken önemli özellikleri belirleyen ve mimari kararları doğrudan etkileyen sistem gereksinimleridir.
- Yalnızca bir yazılımın nasıl inşa edileceğiyle değil, aynı zamanda sistemin uzun vadede nasıl çalışacağıyla ilgilidir.
- Bir sistemin teknik yapısını ve işleyişini etkileyen en önemli unsurlardan biridir.



Diğer İsimler

- **İşlevsel olmayan gereksimler (Nonfunctional Requirements - NFRs)** terimi de yazılım mühendisliğinde mimari karakteristikleri tanımlamak için kullanılabilir.
 - İşlevsel gereksinimler (FRs), sistemin "ne yapması gerektiğini" tanımlar.
 - İşlevsel olmayan gereksinimler (NFRs), sistemin "nasıl çalışması gerektiğini" belirler.
 - Bir e-ticaret sitesinde FR: Kullanıcılar ürün satın alabilmelidir.
 - NFR: Satın alma işlemi 2 saniyeden kısa sürede tamamlanmalıdır (Performans).
- Ancak "Nonfunctional" kelimesi, işlevsel gereksinimlerden daha az önemliymiş gibi bir algı yaratıyor.
- Oysa, performans, güvenlik, ölçeklenebilirlik gibi özellikler bir yazılım sisteminin başarısını doğrudan etkileyen kritik unsurlardır.



Diğer İsimler

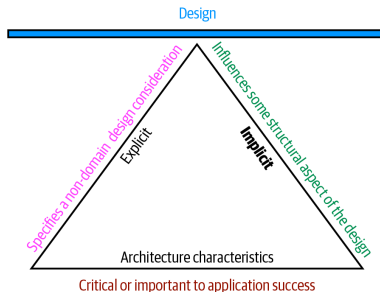
- **Kalite Nitelikleri (Quality Attributes - QAs)** terimi de bir diğer isimdir.
 - Kalite nitelikleri tasarım aşamasında değil de, sonradan yapılan kalite değerlendirmelerini çağrıştırıyor.
 - Halbuki, bu özellikler mimariyi en baştan şekillendirmeli, sonradan eklenmeye çalışılan iyileştirmeler olmamalıdır.
- **Mimari karakteristikler** terimi şunları tarif edebildiği için daha uygun:
 - ✓ Yazılımın işlevsel alanından (problem domain) bağımsız olan tasarım kararlarını tanımlar.
 - ✓ Sistemin yapısal kararlarını etkiler.
 - ✓ Uygulamanın başarısı için kritik olan konuları kapsar.



Mimari Karakteristikler

Mimari karakteristikler şu üç kriteri sağlamalı;

- Sistemin işlevselliğine özgü olmayan bir tasarım konusunu belirtir.
- Tasarımın bazı yapısal yönlerini etkiler.
- Uygulamanın başarısı için kritik veya önemlidir.



Üç kriter - 1

Sistemin işlevselliğine özgü olmayan bir tasarım konusunu belirtir.

- Mimari karakteristikler, sistemin işlevselliğiyle (*domain*) değil, sistemin nasıl çalıştığıyla ilgilidir.
- Örneğin, bir e-ticaret sitesinin "*ürün satın alma*" özelliği bir işlevsel gereksinimdir, ancak **yüksek erişilebilirlik (high availability)** mimari bir karakteristiklerdir.
- **Örnek:**
 - **Yanlış Örnek:** "*Sistem, kullanıcıların bir ürünü sepetine eklemesine izin vermeli.*"
→ Bu bir iş gereksinimidir, mimari karakteristik değil.
 - **Doğru Örnek:** "*Sistem, saniyede 1000 eşzamanlı kullanıcıyı desteklemeli.*"
→ Bu bir mimari karakteristiklerdir.



Üç kriter - 2

Tasarımın bazı yapısal yönlerini etkiler.

- **Mimari karakteristikler**, sistemin yapısal tasarımını etkileyen özel gereksinimlerdir.
- **Genel güvenlik önlemleri**, her sistemde uygulanmalıdır ancak mimari karakteristik seviyesine çıkması, özel bir yapısal düzenleme gerektirip gerektirmediğine bağlıdır.

Örnek:

- **Üçüncü Taraf Ödeme İşlemcisi**
 - Ödeme işlemleri harici bir sağlayıcı tarafından yönetilir.
 - Standart güvenlik önlemleri (şifreleme, hashing) yeterlidir.
 - Özel bir yapısal düzenleme gerektirmez.
- **Uygulama İçinde Ödeme İşleme**
 - Ödeme işlemleri doğrudan uygulama içinde gerçekleşir.
 - Kritik güvenlik risklerini izole etmek için özel bir modül, bileşen veya servis tasarlanmalıdır.
 - Mimari tasarımı doğrudan etkileyen bir karakteristiklerdir.



Üç kriter - 3

Uygulamanın başarısı için kritik veya önemlidir.

- Bazı **mimari karakteristikler** olmadan sistem başarılı olamaz.
- Sistemin **performansını, sürdürülebilirliğini ve bakım kolaylığını** doğrudan etkileyen özelliklerdir.

Örnekler:

- **Gerçek zamanlı işlem gerektiren bir finans uygulaması**
 - **Düşük gecikme süresi** (*low latency*) kritik bir karakteristiktir.
- **Bir sağlık sistemi**
 - **Yüksek güvenilirlik** (*high reliability*) olmazsa olmazdır.



Operasyonel Mimari Karakteristikler

Terim	Tanım
Erişilebilirlik	Sistemin ne kadar süre kullanılabilir olması gerektiği (Örn: 7/24 çalışma gereksinimi).
Süreklilik	Felaket durumlarında sistemin nasıl kurtarılacağı (disaster recovery).
Performans	Stres testi, tepki süreleri, kapasite analizi gibi performans gereksinimleri.
Kurtarılabilirlik	Bir felaket sonrası sistemin ne kadar hızlı tekrar çalışması gerektiği.
Güvenilirlik / Emniyet	Sistemin hata toleransı ve kritik durumlarda güvenli çalışabilme yeteneği.
Sağlamlık	Sistem hata ve sınır durumlarını ne kadar iyi yönetebilir? (Örn: ağ kesintisi, donanım arızası).
Ölçeklenebilirlik	Kullanıcı veya işlem sayısı arttığında sistemin buna nasıl adapte olduğu.



Yapısal Mimari Karakteristikler

Terim	Tanım
Yapılandırılabilirlik	Kullanıcıların yazılımın ayarlarını kolayca değiştirebilmesi.
Geniştirilebilirlik	Yeni işlevlerin sisteme kolayca eklenebilmesi.
Kurulabilirlik	Sistemin gerekli tüm platformlara kolayca kurulabilmesi.
Yeniden Kullanılabilirlik	Ortak bileşenlerin farklı ürünlerde tekrar kullanılabilmesi.
Yerelleştirme	Çoklu dil, farklı para birimleri ve ölçü birimlerini destekleme.
Bakım Kolaylığı	Sistemde değişiklik yapmanın ve geliştirme eklemenin ne kadar kolay olduğu.
Taşınabilirlik	Sistem farklı platformlarda çalışabilir mi? (Örn: Oracle ve SAP DB ile uyumlu olmalı mı?)
Desteklenebilirlik	Hata ayıklama ve teknik destek için gereken seviyeler.
Güncellenebilirlik	Önceki sürümlerden yeni sürümlere kolayca yükseltilebilme.



Kesitler Arası Mimari Karakteristikler

Terim	Tanım
Erişilebilirlik	Engelli kullanıcılar (renk körlüğü, işitme kaybı vb.) için erişim desteği.
Arşivlenebilirlik	Verilerin belirli bir süre sonra silinmesi veya arşivlenmesi gerekip gerekmediği.
Kimlik Doğrulama	Kullanıcıların gerçek kimliklerini doğrulama gereksinimi.
Yetkilendirme	Kullanıcıların yalnızca belirli işlemlere erişebilmesi (sayfa, işlem, iş kuralı vb. bazında).
Yasal Uyumluluk	Sistem için geçerli olan yasal düzenlemeler (KVKK vb.).
Gizlilik	Dahili çalışanların belirli işlemleri görememesini sağlama (şifrelenmiş işlemler vb.).
Güvenlik	Veritabanında ve ağ iletişiminde veri şifreleme gereksinimi.
Desteklenebilirlik	Hata ayıklama ve teknik destek için gereken seviye ve loglama gereksinimleri.
Kullanılabilirlik	Kullanıcıların sistemi verimli kullanabilmesi için gereken eğitim seviyesi.



Mimari karakteristikleri seçmek

- Bir uygulama çok sayıda **mimari karakteristik** destekleyebilir... ama bu her zaman iyi bir şey değildir.
- Her eklenen mimari karakteristik, **tasarımın karmaşıklığını artırır.**
- Bu nedenle, bir mimarın en önemli görevlerinden biri, **en fazla özelliği eklemek değil, en gerekli olanları seçmektir.**



Bazı Terimler Özel ve Muğlak Olabilir

- **Interoperability (birlikte çalışabilirlik)** ve **compatibility (uyumluluk)** benzer görünebilir.
- Ancak **interoperability**, sistemin diğer sistemlerle entegrasyonunun kolaylığına ve API dokümantasyonuna odaklanır.
- **Compatibility** ise sektör ve alan standartlarına uygunluğu vurgular.



Aynı Terim Farklı Şekillerde Yorumlanabilir

- **Learnability (öğrenilebilirlik):**

- Bir kullanıcının yazılımı öğrenme kolaylığı anlamına gelebilir.
- Sistemin çevresini analiz ederek kendini optimize etmesi anlamına da gelebilir.



Bazı Tanımlar Birbirine Yaklaşabilir ama Aynı Değildir

- **Availability (kullanılabilirlik)** ve **reliability (güvenilirlik)** benzer kavramlardır ama her zaman aynı şeyi ifade etmezler.
- Örnek: **UDP (User Datagram Protocol)** ağ üzerinden her zaman erişilebilir (available) olabilir ancak güvenilir (reliable) değildir.
- UDP'de paketler eksik gelebilir veya sırası bozulabilir.



Mimari karakteristikler

- Mimari karakteristikleri değerlendirirken **terimlerin bağlamını iyi anlamak** gerekir.
- Belirsizliği minimize etmek için net tanımlar tercih edilmelidir.
- Mimarların bu tür kavramları dikkatlice ele alması önemlidir.
- Asla en iyi (best) mimariyi değil, en az kötü (least worst) mimariyi hedefleyin.



Mimari Karakteristiklerin Belirlenmesi

- Mimari tasarımın ilk adımlarından biri, sistem için kritik olan mimari karakteristikleri belirlemektir.
- Bu karakteristiklerin doğru belirlenmesi, yalnızca teknik bilgiyi değil, aynı zamanda iş alanının gereksinimlerini anlamayı da gerektirir.
- "-ility" özellikleri (Scalability, Availability, Maintainability vb.) iş alanına özgü olarak seçilmelidir.



Doğru Mimari Karakteristikleri Belirleme

- Mimari kararlar sadece teknik bir süreç değil, iş gereksinimlerini de dikkate almayı gerektirir.
- Alan uzmanları ve paydaşlarla iletişim kurarak, sistemin gerçekten hangi mimari niteliklere ihtiyacı olduğu belirlenmelidir.
- Mimarlar, mimari karakteristikleri üç temel kaynaktan çıkarır:
 - **Alan (Domain) Kaygıları** → İş süreçleri ve sektör standartlarından gelen ihtiyaçlar.
 - **Gereksinimler (Requirements)** → Proje dökümanlarında açıkça belirtilen teknik ve iş ihtiyaçları.
 - **Örtük Bilgi (Implicit Knowledge)** → Açıkça belirtilmeyen, ancak sektörde tecrübeye dayalı olarak bilinen kritik faktörler.



Alan (Domain) Kaygıları

Basit bir e-ticaret platformu örneği üzerinden bu üç kaynağı ele alalım:

- **Ödeme Güvenliği (Security)**
Kullanıcıların kredi kartı bilgileri şifrelenmeli.
- **Yüksek Erişilebilirlik (Availability)**
Kullanıcılar günün her saatinde alışveriş yapabilmeli.
- **Ölçeklenebilirlik (Elasticity)**
Black Friday gibi yoğun zamanlarda sistem çökmemeli.

Örnek: Ödeme sistemlerinde *PCI DSS* gibi standartlar bulunur, bu yüzden güvenlik kritik bir mimari karakteristiktir.



Gereksinimler (Requirements)

- **Sistem, 5000 eşzamanlı kullanıcıyı desteklemeli.**
(*Scalability*)
- **Siparişler, ödeme onaylandıktan sonra 2 saniye içinde işlenmeli.**
(*Performance*)
- **Sistemin yıllık çalışma süresi %99.9 olmalı.**
(*Availability ve Reliability*)

Örnek: Eğer ödeme işlemi çok uzun sürerse, müşteri deneyimi olumsuz etkilenir. Performans burada kritik bir karakteristiktir.



Örtük Bilgi (Implicit Knowledge)

- **Kampanya Dönemlerinde Trafik Artışı (Scalability)**

Yoğun dönemlerde sistem buna hazırlıklı olmalı.

- **Müşteri Destek Sistemi (Supportability)**

Kullanıcılar ödeme hataları yaşarsa, hata yönetim mekanizması olmalı.

- **SEO Dostu Mimari (Performance ve Usability)**

Sayfalar hızlı yüklenmeli ve SEO için optimize edilmeli.

Örnek: Resmi gereksinimlerde SEO belirtilmese de, hızlı sayfa yüklenmesi ve optimize edilmiş URL yapıları rekabet avantajı sağlar.



Sonuç - Mimari Karakteristiklerin Belirlenmesi

- **Domain Kaygılarından** → *Security, Availability, Scalability*
- **Gereksinimlerden** → *Performance, Availability, Scalability*
- **Örtük Bilgiden** → *Scalability, Supportability, Performance*

Sonuç: Bu üç faktör bir araya geldiğinde, mimari tasarımın yönü belirlenmiş olur.



Aşırı Mimari Karakteristikler ve Karmaşıklık

- Çok sık yapılan bir anti-desen de her karakteristiği destekleyecek jenerik bir mimari tasarlamaktır.
- Her mimari karakteristiği desteklemek, sistemin karmaşıklığını artırır.
- Fazla özellik eklemek, mimariyi gereksiz yere zorlaştırır ve yönetilemez hale getirebilir.
- Mimarlar, yalnızca gerçekten gerekli olan özelliklere odaklanmalıdır.



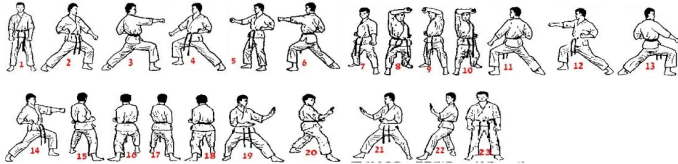
Vasa Gemisi: Aşırı Tasarımın Felaketi

- İsveç Kralı Adolphus, hem ‘taşıma hem de savaş gemisi’ olarak kullanılacak eşsiz bir gemi istedi.
- Tek güverte yerine çift güverte, normalin iki katı ağırlığında toplar vs. gibi gereksiz Karakteristikler eklendi.
- Limandan çıkarken tek taraflı top atışı yaptı ve dengesini kaybederek **battı**.
- Aşırı gereksinimler ve karmaşıklık, geminin işlevselliğini yok etti.



Mimari Katarlar (Architecture Katas) Nedir?

- Mimari kata, mimari tasarım pratiği yapmayı sağlayan bir yöntemdir.
- Terim, Japon dövüş sanatlarındaki kata kavramından gelir ve bireysel teknik çalışmalarına benzer bir anlama sahiptir.
- Ted Neward, mimarların nadiren mimari tasarım yapma fırsatı bulduğunu fark ederek bu yöntemi geliştirmiştir.



Kata Egzersizi Nasıl Yapılır?

‘How do we get great designers? Great designers design, of course.’

— Fred Brooks

‘So how are we supposed to get great architects, if they only get the chance to architect fewer than a half-dozen times in their career?’

— Ted Neward

- Küçük ekipler, verilen bir alan problemine ve ek bağlam bilgilerine göre bir mimari çözüm tasarlar.
- Çalışmalarını diğer gruplara sunar ve en iyi mimariyi belirlemek için değerlendirme yapılır.



Kata Yapısının Temel Bölümleri

- **Tanım:** Çözülmesi gereken genel alan problemi.
- **Kullanıcılar:** Sistemden beklenen kullanıcı sayısı veya türleri.
- **Gereksinimler:** Alan uzmanlarından beklenen iş ve teknik gereksinimler.
- **Ek Bağlam:** Açıkça belirtilmemiş, ancak tasarımı etkileyen faktörler.



Mimari Kataların Faydaları

- Gerçek dünyaya yakın senaryolarla deneyim kazanmayı sağlar.
- Mimarların tasarım yetkinliklerini geliştirmelerine yardımcı olur.
- Takım çalışması ve değerlendirme süreçleriyle farklı bakış açıları kazandırır.

Mimari Katalar

<https://www.architecturalkatas.com/>



Vaka Çalışması: Silicon Sandwiches



Tanım:

- Ulusal çapta hizmet veren bir sandviç dükkanı, mevcut telefonla sipariş hizmetine ek olarak çevrimiçi sipariş alabilmek istiyor.

Kullanıcılar:

- Binlerce, belki de ileride milyonlarca kullanıcı.



Gereksinimler

- Kullanıcılar sipariş verdikten sonra, kendilerine bir teslim alma saati ve dükkana giden yol tarifi sağlanmalıdır.
- Trafik bilgisi içeren çeşitli harita servisleriyle entegrasyon gereklidir.
- Eğer dükkanda teslimat hizmeti varsa, sipariş teslim edilmek üzere bir kuryeye yönlendirilmelidir.
- Mobil cihazlardan erişilebilir olmalıdır.
- Ulusal çapta günlük kampanya ve indirimler sunulmalıdır.
- Yerel düzeyde günlük kampanya ve indirimler sunulmalıdır.
- Ödeme, çevrimiçi, teslim alma sırasında veya teslimat sırasında yapılabilmelidir.



Ek Bağlam

- Sandviç dükkanları, her biri farklı bir sahibi olan franchise işletmeleridir.
- Ana şirketin yakın gelecekte yurt dışına açılma planları bulunmaktadır.
- Şirketin kurumsal hedefi, kârı maksimize etmek için düşük maliyetli iş gücü kullanmaktır.



Silicon Sandwiches Kata Çalışması

Bu kata çalışması, bir mimarın gereksinimlerden mimari karakteristikleri nasıl türeteceğini gösterir.

- Ulusal bir sandviç dükkanı çevrimiçi sipariş sistemi eklemek istiyor.
- Kullanıcı sayısı binlerce, gelecekte milyonlara ulaşabilir.



Açık (Explicit) Mimari Karakteristikler

- **Ölçeklenebilirlik (Scalability):** Kullanıcı yükünün zamanla artması.
- **Esneklik (Elasticity):** Ani kullanıcı artışlarına dayanabilme.
- **Özelleştirilebilirlik (Customizability):** Kampanya ve trafik bilgilerinin kişiselleştirilmesi.
- **Uluslararasılaştırma:** Farklı diller ve para birimlerine destek.
- **Güvenlik (Security):** Çevrimiçi ödeme için temel güvenlik önlemleri.
- **Mobil Uyumlu Tasarım:** Responsive web veya mobil uygulama ihtiyacı.
- **Gerçeklenebilirlik (Feasibility):** Düşük maliyetli çözümler.



Örtük (Implicit) Mimari Karakteristikler

- **Kullanılabilirlik (Availability):** Kullanıcıların siteye her zaman erişebilmesi.
- **Güvenilirlik (Reliability):** Sipariş sırasında bağlantı kopmalarının önlenmesi.
- **Güvenlik (Security):** Kritik bir güvenlik ihtiyacı olmamakla birlikte temel önlemler gerekli.
- **Özelleştirilebilirlik (Customizability):** Franchise'lar bazında özelleştirmeler olabilir.



Mimari Tasarım Sürecindeki Kararlar

- Mikroçekirdek mi, geleneksel tasarım mı?
 - Mikroçekirdek yüksek özelleştirilebilirlik sunar, ancak performans ve bağımlılıkları etkileyebilir.
 - Alternatif olarak Template Method gibi tasarım desenleriyle de özelleştirme sağlanabilir.
- Maliyet ve karmaşıklık analizi yapılmalı.
 - Hangi çözüm, hedeflenen mimari karakteristikleri daha düşük maliyetle sağlar?
- İşbirliği önemlidir!
 - Mimar, geliştiriciler, proje yöneticileri ve iş analistleri ile birlikte karar vermelidir. Ivory Tower'dan kaçınmak gerek.



Sorular.



Bölüm 5

Mimari Karakteristikleri Ölçme ve Yönetme



Mimari Karakteristikleri Ölçme ve Yönetme

Mimarlar, yazılım projelerinde çok çeşitli mimari karakteristikleri yönetmek zorundadır:

- Operasyonel karakteristikler: **Performans, Esneklik, ölçeklenebilirlik**
- Yapısal karakteristikler: **Okunabilirlik, yayınlanabilirlik**

Bu bölüm, yaygın mimari karakteristikleri tanımlamaya ve onları yönetmek için mekanizmalar oluşturmaya odaklanmaktadır.



Mimari Karakteristikleri Ölçerken Karşılaşılan Sorunlar

- **Fiziksel Olmayan Kavramlar:** - Çeviklik (*agility*) veya yayınlanabilirlik (*deployability*) gibi soyut kavramlar için net bir tasarım yolu yoktur.
- **Değişken Tanımlar:** - Aynı organizasyonda bile, farklı ekipler kritik özellikleri farklı tanımlayabilir (örneğin *performans*).
- **Çok Bileşenli Yapılar:** - Örneğin, çeviklik (*agility*) aslında modülerlik (*modularity*), yayınlanabilirlik (*deployability*) ve test edilebilirlik (*testability*) gibi alt özelliklerden oluşur.



Çözüm: Objektif Tanımlar

Proje ekibi genelinde ortak tanımlar belirlemek, üç temel fayda sağlar:

❶ Ortak mimari dili oluşturma:

- Tüm ekiplerin aynı terimleri aynı şekilde anlamasını sağlar.

❷ Ölçülebilirlik:

- Karakteristikleri somut verilerle değerlendirmeye yardımcı olur.

❸ Bileşenleri ayrıştırma:

- Büyük kavramları küçük, ölçülebilir alt bileşenlere ayırarak yönetilebilir hale getirir.



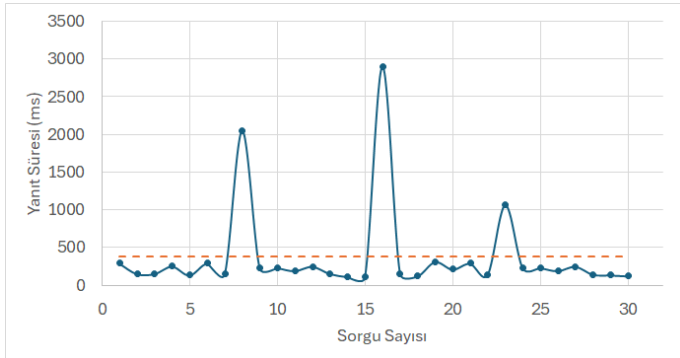
Operasyonel Ölçümler

- Bazı mimari karakteristikler doğrudan ölçülebilir:
 - Performans
 - Ölçeklenebilirlik gibi
- Ancak, ölçüm yöntemleri kritik öneme sahiptir. Örneğin, Bir sorguya ait yanıt süresi performans ölçütü olarak incelenecekse;
 - Ortalama yanıt süresi odağa alınırsa hatalı bir analiz yapılmış olabilir.
 - Çünkü uç değerler (outliers) gözden kaçırılabilir.
 - Bu yüzden maksimum yanıt süresi ölçümü, aşırı gecikmeleri tespit etmek için önemlidir.



Operasyonel Ölçümler

- Örnek bir sorgu yanıt süresi analizi:



- Sadece ortalama değeri (374,67 ms) baz alırsak hatalı bir analiz yapmış oluruz.
- Uç değerleri (outliers) gözden kaçırmamak lazım.



İstatistiksel Analiz ve Dinamik Ölçüm

- Profesyonel ekipler sabit eşikler yerine istatistiksel modeller kullanmalıdır.
- Örnek: Bir video akış hizmeti ölçeklenebilirliği nasıl yönetir?
 - Ölçeklenebilirlik zaman içinde izlenir ve istatistiksel modeller oluşturulur.
 - Gerçek zamanlı veriler modelin dışına çıktığında alarm üretilir.
- Bu yöntem, hem model hatalarını hem de sistemdeki anormallikleri tespit etmeye yardımcı olur.



İstatistiksel Yöntemlerle Ölçklenebilirlik Takibi

- **Hareketli Ortalamalar:** Son n ölçümün ortalamasını alarak ani değişimlere odaklanabilir.
- **Z-Skoru ile Anomali Tespiti:** Ölçümün ortalamadan kaç standart sapma uzaklıkta olduğunu belirler. $\pm 3\sigma$ dışındaki değerler anomali kabul edilebilir.
- **EWMA (Üstel Ağırlıklı Hareketli Ortalama):** Güncel verilere daha fazla ağırlık vererek ani değişimlere duyarlıdır.
- **ARIMA (AutoRegressive Integrated Moving Average) ile Tahminleme:** Geçmiş verilere dayanarak gelecekteki yük tahmin edilir, sapmalar alarm tetikleyebilir.
- **Yüzdelik (Percentile) Takibi:** p95 veya p99 değerlerini izleyerek en kötü durum performansı değerlendirilir.



Yeni Ölçüm Yöntemleri ve Gelişen Metrikler

- Ölçülebilir karakteristikler sürekli gelişmektedir.
- Örnek: Web performansı için yeni metrikler:
 - İlk içeriğin yüklenme süresi (First Contentful Paint)
 - İlk işlem için CPU'nun hazır olma süresi (First CPU Idle)
- Cihazlar, hedefler ve teknolojiler değiştikçe yeni ölçüm yöntemleri geliştirilmektedir.



Yapısal (Structural) Ölçümler

Yapısal ölçümler, kod kalitesini değerlendirmek için kullanılan metriklerdir:

- **Çeviklik ve Modülerlik**

- Modüler bir sistem, bağımlılığı en az bileşenlerden oluşmalıdır.
- **Bağımlılık Yoğunluğu:** Bir modülün diğer modüllere olan bağımlılığını ölçerek esneklik ve değiştirilebilirlik değerlendirilir.

- **Bağımlılık ve Bağlılık (Coupling and Cohesion)**

- **Bağımlılık (Coupling):** Bir bileşenin başka bileşenlere bağımlılık seviyesini gösterir. Yüksek bağımlılık kod değişikliklerini zorlaştırır.
- **Bağlılık (Cohesion):** Bir bileşenin içindeki parçaların birbirleriyle ilişkisini gösterir. Yüksek bağlılık, bileşenin belirli bir işlevi iyi yaptığını gösterir.



Yapısal (Structural) Ölçümler

Yapısal ölçümler, kod kalitesini değerlendirmek için kullanılan metriklerdir:

- **Tekrarlılık (Duplication) ve Anlaşılabilirlik**

- **Tekrarlayan Kod:** Aynı işlevin farklı yerlerde tekrarlanması kötü bir işarettir.
- **Anlaşılabilirlik:** Yorum satırları/ kod oranı, fonksiyon uzunluğu, değişken adları gibi faktörler kodun okunabilirliğini etkiler.

- **Kodun Karmaşıklığı**

- Aşırı karmaşık kod, kod kalitesini olumsuz etkileyebilir.
- **Siklomatik Karmaşıklık:** Bir fonksiyonun sahip olduğu bağımsız yürütme yollarını sayarak kodun test edilebilirliğini ve hata yapma olasılığını belirler.



Siklomatik Karmaşıklık Nedir?

- Siklomatik karmaşıklık, bir programın veya fonksiyonun kontrol akış grafiğini kullanarak hesaplanan bir ölçüdür.
- Temel olarak, kodun ne kadar karmaşık olduğunu belirler ve test edilmesi gereken bağımsız yürütme yollarının sayısını gösterir.
- Bu metrik, Thomas McCabe tarafından 1976 yılında önerilmiş olup, kodun anlaşılabilirliğini, hata yapma olasılığını ve test edilebilirliğini analiz etmek için kullanılır.



Bir fonksiyon veya programın akış grafiği çıkarılır ve aşağıdaki formülle siklomatik karmaşıklık hesaplanır:

$$CC = E - N + 2$$

Burada:

- **E (Edge - Kenar):** Akış grafiğindeki yönlü bağlantıların (okların) sayısıdır.
- **N (Node - Düğüm):** Akış grafiğindeki karar noktalarının (örneğin if, while, for gibi kontrol ifadelerinin) ve işlem düğümlerinin toplam sayısıdır.

Bazı kaynaklar, CC'yi P (programın birbirine bağlı bileşenlerinin sayısı) kullanarak şu şekilde de hesaplar:

$$CC = E - N + 2P$$

Eğer tek bir fonksiyon veya bağımsız bir kod parçası analiz ediliyorsa, P genellikle 1 alınır.

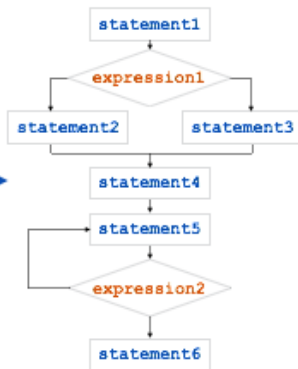


Siklomatik Karmaşıklık

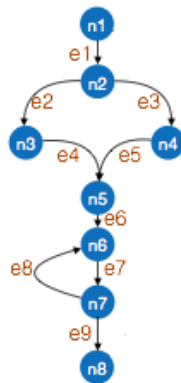
Code

```
statement1  
If expression1  
    statement2  
else  
    statement3  
statement4  
do  
    statement5  
while expression2  
statement6
```

Flow-Chart



Flow-Graph



Siklomatik Karmaşıklık

CC Değeri	Yorum
1-10	Düşük karmaşıklık, kod iyi yönetilebilir.
10-20	Orta seviye karmaşıklık, kod gözden geçirilmeli.
20-40	Yüksek karmaşıklık, refaktör edilmesi önerilir.
>40	Kritik seviyede karmaşıklık, kod çok zor test edilir.



Bölüm 6

Bileşen Tabanlı Düşünme

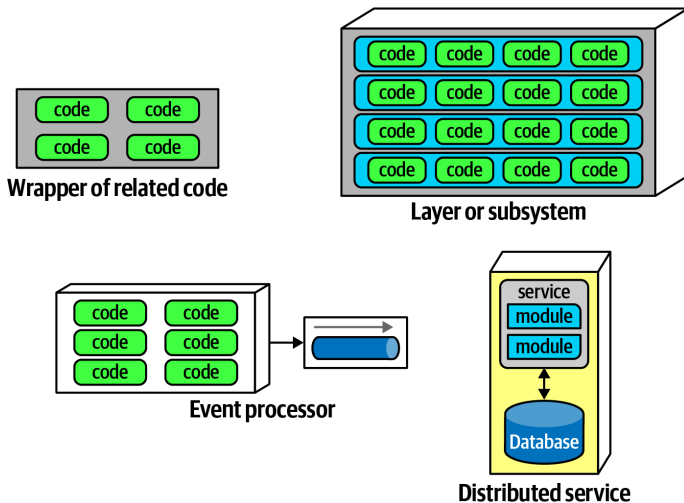


Modül ve Bileşen Nedir?

- Yazılım mimarisinde, modül ve bileşen kavramları sıkça birlikte anılır ancak farklı anlamlara gelir.
- **Modül (Module):** Derleme (build time) sırasında anlamlı olan, ilişkili kodları mantıksal olarak gruplayan yapıdır.
- **Bileşen (Component):** Modülün fiziksel olarak paketlenmiş ve çalışma zamanında (run time) kullanılabilen halidir.
- Farklı dillerde farklı biçimlerde somutlaşır:
 - Java: .jar
 - .NET: .dll
 - Ruby: .gem
- Mimarlar, sistemin modüllerini nasıl bileşenlere dönüştüreceklerini planlayarak çalıştırılabilir, dağıtılabilir bir yapı oluştururlar.



Bileşen Kapsamı ve Türleri



Bileşen Kapsamı ve Türleri

- **Kütüphane (Library):**

- Aynı adres alanında çalışır.
- Fonksiyon çağrısı üzerinden iletişim kurar.
- Genellikle derleme zamanında sisteme dahil edilir.

- **Servis (Service):**

- Kendi adres alanında bağımsız çalışır.
- Mikroservis mimarisinde yaygın olarak kullanılır.

- **Alt Sistem / Katman:**

- Birden fazla bileşenin bir araya gelerek oluşturduğu üst yapılar olabilir.
- Mimaride modülerliğin bir başka düzeyini oluşturur.



Neden Bileşen Bazlı Düşünmeliyiz?

- Bileşenler, mimari düşünmenin temel yapı taşlarıdır.
- Kodun daha üst düzeyde organizasyonunu sağlar; karmaşıklığı sınırlandırır.
- Her bileşen, ayrı geliştirilebilir, test edilebilir ve dağıtılabılır hale gelir.
- Mikroservis gibi modern mimarilerde her bir servis bir bileşen olarak düşünülür.
- Bileşen bazlı yapı, ekiplerin bağımsız çalışabilmesine ve sistemin ölçeklenebilirliğine katkı sağlar.
- Bileşen sınırlarının net tanımlanması, sistemin evrimsel gelişimini kolaylaştırır.



Mimarın Rolü

- Yazılım mimarı, sistemin bileşenlerini tanımlar ve nasıl bir araya geleceğini planlar.
- Bu işi tek başına değil, farklı ekiplerle birlikte yapar:
 - Geliştiriciler
 - İş analistleri
 - Testçiler
 - Operasyon ekipleri
- Hedef: Yazılımın hem işlevsel hem de mimari gereksinimlerini karşılayan bir yapı kurmak.



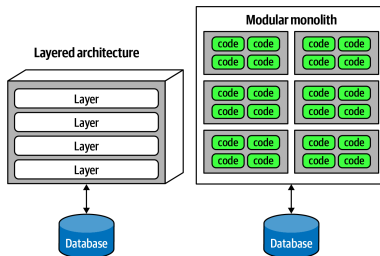
Sorumluluklar ve Sınırlar

- Mimar, genelde sınıf veya fonksiyon düzeyine değil, bileşen düzeyine odaklanır.
- Tasarım kalıpları gibi konularda teknik katkı verebilir, ama detaylara boğulmamalı.
- Ekip üyelerinin kendi kararlarını almasına alan tanımalı.
- Aksi halde ekip bağımsız hareket edemez, mimari liderlik gelişmez.



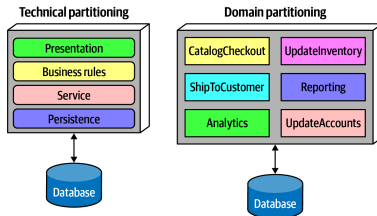
Mimari Üst-Seviye Bölümlendirme

- Yazılım mimarisini organize etmenin farklı yolları vardır.
- Üst seviyedeki bölümlendirme, mimarinin nasıl yapılandırılacağına dair önemli bir karardır.
- İki ana yaklaşımdan örnek verelim:
 - **Katmanlı Mimari (Layered Architecture):** Tekniksel katmanlara dayalı yapılandırma.
 - **Alan Bazlı Mimari (Domain-Based Architecture):** İş süreçlerine ve alanlara dayalı yapılandırma.



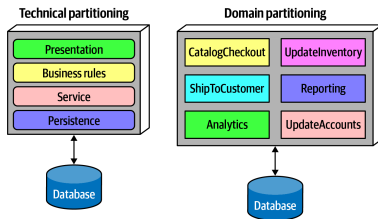
Katmanlı Mimari (Layered Architecture)

- Katmanlı mimari, sistemin fonksiyonlarını belirli teknik işlevlere böler.
- Yaygın olarak kullanılan katmanlar:
 - Sunum Katmanı
 - İş Kuralları Katmanı
 - Servis Katmanı
 - Kalıcılık Katmanı (Persistence)
- Bu yapının avantajı:
 - Kodun organize edilmesi ve belirli alanlara ayrılması kolaylaşır.
 - Her katman belirli bir rolü yerine getirir.



Alan Bazlı Mimari (Domain-Driven Design - DDD)

- Alan bazlı mimari, yazılımı bağımsız alanlara veya iş akışlarına göre böler.
- Her alan, diğerlerinden bağımsız olarak tasarlanır ve daha az bağımlılıkla çalışır.
- Örnek: Mikroservis Mimarisi.
- Avantajı:
 - Her alanın kendi bağımsız işlevselliği vardır, bu da yazılımın modülerliğini artırır.



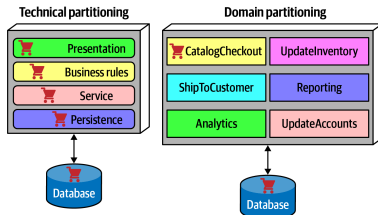
Karşılaştırma: Katmanlı ve Alan Bazlı Mimari

● Katmanlı Mimari:

- Teknik yeteneklere göre organize edilir.
- Daha yüksek seviyeli ayrımlar sağlar.
- Modülerlik ve ayırım, katmanlar arasında güçlüdür.

● Alan Bazlı Mimari (DDD):

- İş akışları veya alanlara göre organize edilir.
- İşlevsel bütünlük ve bağımsızlık sağlar.
- Mikroservislerde yaygın olarak kullanılır.



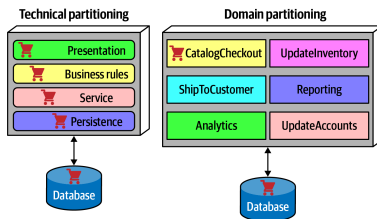
Sonuç: İki Yaklaşımın Uygulama Alanı

● Katmanlı Mimari:

- Uygulama, net bir teknik yapı gerektiriyorsa uygun.
- Kolay anlaşılabilir ve belirli bir yazılımın her yönü için uygun.

● Alan Bazlı Mimari (DDD):

- İş süreçleri veya domain'lere dayalı sistemler için uygun.
- Bağımsız ve esnek yapılar gerektiren durumlar için idealdir.



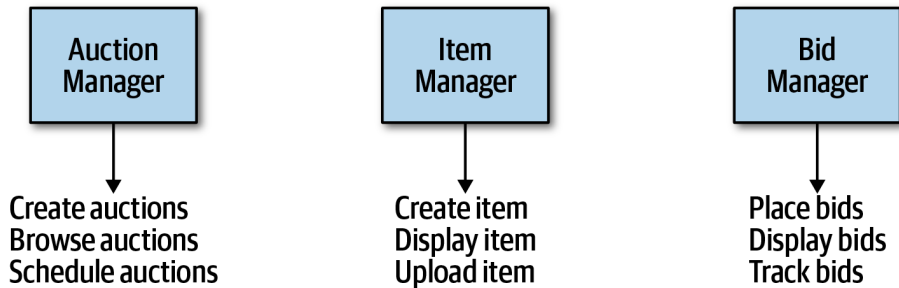
Conway'in Kanunu

- **Conway'in Kanunu:** "Bir organizasyon, tasarladığı sistemin yapısını, organizasyonel yapısına benzer şekilde inşa eder."
- Bu kanun, organizasyon yapısının yazılım mimarisi üzerinde doğrudan bir etkisi olduğunu belirtir.
- Eğer bir yazılım mimarisi katmanlı bir yapıya sahipse, organizasyonun da bu katmanlara uygun bir şekilde yapılandırılması beklenir.
- **Örnek:**
 - Katmanlı mimari: Yazılımda katmanlar oluşturulmuşsa, organizasyonun farklı ekipleri bu katmanlara göre ayrılabilir (veritabanı, iş mantığı, kullanıcı arayüzü vb.).
 - Alan bazlı mimari: İş süreçlerine dayalı bir yapı varsa, organizasyon da bu alanlara uygun şekilde organize olabilir.



Bileşen Tasarımı

Entity Trap (Varlık Tuzağı) Nedir?



Bileşen Tasarımı

Entity Trap (Varlık Tuzağı) Nedir?

- Sistemi, veri yapıları (örneğin: Customer, Order, Product) etrafında tasarlama eğilimidir.
- Bileşenler davranış yerine veri modellerine göre oluşturulur.
- Genellikle şu şekilde görülür: Her entity için bir service yazılır ama iş akışları dağılır.



Bileşen Tasarımı

Entity Trap (Varlık Tuzağı) Nedir?

- Sistemi, veri yapıları (örneğin: Customer, Order, Product) etrafında tasarlama eğilimidir.
- Bileşenler davranış yerine veri modellerine göre oluşturulur.
- Genellikle şu şekilde görülür: Her entity için bir service yazılır ama iş akışları dağınık.

Neden Sakıncalıdır?

- Ayırık işlevleri olan entity'ler karmaşık hale gelir (*Tanrı Sınıfı problemi*).
- Davranışlar veri yapılarından ayrılmadığı için değişiklikler tüm sistemi etkiler.
- Gerçek iş süreçlerini yansıtmakta yetersiz kalır.



Bileşen Tasarımı

Entity Trap (Varlık Tuzağı) Nedir?

- Sistemi, veri yapıları (örneğin: Customer, Order, Product) etrafında tasarlama eğilimidir.
- Bileşenler davranış yerine veri modellerine göre oluşturulur.
- Genellikle şu şekilde görülür: Her entity için bir service yazılır ama iş akışları dağınık.

Neden Sakıncalıdır?

- Ayırık işlevleri olan entity'ler karmaşık hale gelir (*Tanrı Sınıfı problemi*).
- Davranışlar veri yapılarından ayrılmadığı için değişiklikler tüm sistemi etkiler.
- Gerçek iş süreçlerini yansıtmakta yetersiz kalır.

Alternatif: Bileşenleri **eylem**, **ihtiyaç** veya **iç süreçler** üzerinden modellemek (örneğin: CheckoutProcessor, InvoiceGenerator)



Entity Trap: Açık Artırma Örneği

- Bileşenler: `AuctionManager`, `ItemManager`, `BidManager`
- Her bileşen yalnızca kendi verisi (entity) üzerine odaklanmış.
- Davranışlar bu bileşenler arasında dağılmış durumda.
- Süreç akışı bütüncül bir şekilde izlenemiyor.
- Bu yapı test, bakım ve genişletme açısından zorluk çıkarır.



Actor/Action Yaklaşımı

- Bileşenler, sistemdeki aktörlerin gerçekleştirdiği eylemler etrafında şekillenir.
- Örnekler:
 - `BidService`: Kullanıcının teklif verme işlemini yürütür.
 - `AuctionCloser`: Açık artırmayı zamanında kapatır.
 - `CheckoutHandler`: Kazanan kullanıcıdan ödeme alır.
- Rol ve sorumluluklar açık şekilde ayrılır.



Event Storming Yaklaşımı

- Sistem, önemli olaylar (events) etrafında modellenir.
- Örnek olaylar:
 - `BidPlaced`, `AuctionClosed`, `AuctionWon`, `PaymentReceived`
- Her olay bir işlem başlatır veya diğer bileşenlerle bağlantı kurar.
- Gevşek bağlı, genişletilebilir, esnek bir yapı sağlar.



Workflow Bazlı Yaklaşım

- Sistemin akışı, gerçek kullanıcı süreçlerine göre modellenir.
- Örnek bir iş akışı:
 - 1 Ürün incelenir
 - 2 Teklif verilir
 - 3 Teklif kazanılır
 - 4 Ödeme yapılır
- Bileşenler örnek:
 - AuctionWorkflowService, BidSubmissionHandler, WinnerNotificationService
- Süreçler merkezi bir biçimde takip edilebilir.



Yaklaşımların Karşılaştırması

Yaklaşım	Avantajı	Dezavantajı
Entity-based (Trap)	Veritabanı odaklı net yapı	Davranışlar dağınık, süreç takibi zor
Actor/Action	Rol ve görevler net	Entity ilişkileri ihmal edilebilir
Event Storming	Gevşek bağlı, esnek yapı	Event yönetimi karmaşıklaşabilir
Workflow-based	Kullanıcı süreci odaklıdır	Süreç dışı ihtiyaçlar ihmal edilebilir



Sorular.



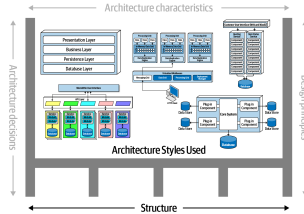
Bölüm 7

Mimari Stiller



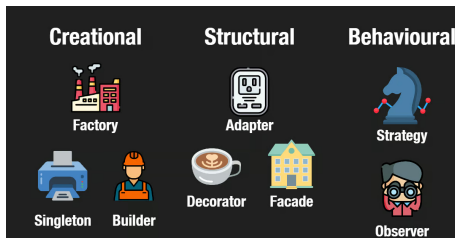
Mimari Stil Nedir?

- **Mimari stil**, bileşenlerin nasıl organize edildiğini ve birbirleriyle nasıl etkileşime geçtiğini tarif eden bir şablondur.
- Bir stil, sistemin topolojisini, mimari karakteristiklerini ve dağıtım modelini etkiler.
- Stil isimleri, mimarlar arasında kısa ve öz bir iletişim sağlar:
 - “Katmanlı monolit” deyince herkes yapıyı, avantajları ve sınırlılıkları anlar.
- Tasarım desenleri gibi, stiller de yıllar içinde biriken deneyimlerin sonucudur.



Mimari Stil ile Tasarım Deseni Arasındaki Fark

- **Mimari Stil:** Sistem düzeyinde organizasyon sağlar.
 - Örnek: Katmanlı mimari, Boruhattı mimarisi
- **Tasarım Deseni:** Stil içinde çözüm kalıpları sunar.
 - Örnek: Observer, Factory, Strategy desenleri
- Stil, mimariyi; desen, mimarinin içini yapılandırır.



Temel Mimari Stiller

- **Big Ball of Mud:** Yapısal bütünlükten yoksun, düzensiz sistemler.
- **Unitary Architecture:** Tek bir işlem içinde çalışan, basit yapılar.
- **Client/Server:** Sunucu ve istemci ayrımı üzerinden iki katmanlı yapı.
- **Three-Tier Architecture:** Sunum, uygulama ve veri katmanı ayrımıyla daha fazla ayrıştırılmış yapı.



Mimari Stillerin Sınıflandırılması

- Mimari stiller genel olarak iki ana kategoriye ayrılır:
 - **Monolitik Mimari Stiller:** Tüm kod tek bir dağıtım birimi içinde yer alır.
 - **Dağıtık Mimari Stiller:** Kod, uzaktan erişim protokolleriyle iletişim kuran birden fazla dağıtım birimine ayrılmıştır.
- Bu ayrım, mimari stiller arasındaki farkları anlamada etkili bir yaklaşımdır.
- Derste detaylı olarak ele alınan mimari stiller:
 - **Monolitik:** Layered, Pipeline, Microkernel
 - **Dağıtık:** Service-based, Event-driven, Space-based, Service-oriented, Microservices



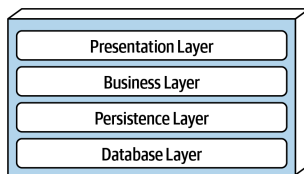
Katmanlı Mimari Stiline Giriş

- **Katmanlı Mimari (Layered Architecture):** En yaygın kullanılan mimari stillerden biridir.
- Sıklıkla **n-tiered architecture** olarak da adlandırılır.
- Yaygın olmasının nedenleri:
 - Basit ve anlaşılır olması
 - Düşük maliyetli olması
 - Geliştirici ekiplerin organizasyon yapısına doğal olarak uyum sağlaması (Conway's Law)
- Katmanlı mimari genellikle şu rollerle örtüşür:
 - Arayüz geliştiricileri (UI developers)
 - İş kuralları geliştiricileri
 - Veritabanı uzmanları (DBA)
- Dikkat: Katmanlı mimari, bazı mimari anti-pattern'lerle de ilişkilidir:
 - Architecture by Implication
 - Accidental Architecture



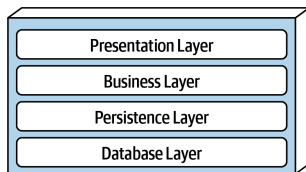
Katmanlı Mimari: Mantıksal Yapı

- Katmanlı mimaride bileşenler yatay katmanlara ayrılır.
- Her katman belirli bir sorumluluk üstlenir.
- En yaygın 4 katman:
 - **Presentation:** Kullanıcı arayüzü ve iletişim mantığı
 - **Business:** İş kuralları ve süreçleri
 - **Persistence:** Veri erişimi ve yönetimi
 - **Database:** Fiziksel veri deposu
- Küçük sistemlerde katmanlar birleştirilebilir (ör: iş ve veri erişim katmanı).
- Büyük sistemlerde 5 veya daha fazla katman olabilir.



Katmanların Görev ve Sorumlulukları

- ❶ **Presentation:** Veri gösterimi, kullanıcıyla etkileşim
 - ❷ **Business:** İş kuralları, karar mekanizmaları
 - ❸ **Persistence:** Verinin alınması, saklanması
 - ❹ **Database:** Verinin fiziksel tutulduğu yer
- Katmanlar birbirlerinin iç yapısını bilmez, sadece ne yaptığını bilir.
 - Bu ayırım sayesinde geliştiriciler uzmanlık alanlarına odaklanabilir.



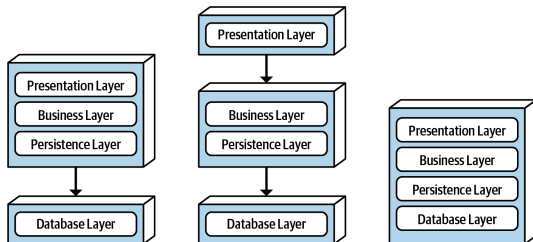
Katmanlı Mimari: Teknik Tabanlı Bölme

- Katmanlı mimari, **teknik sorumluluklara göre** bölme yapar.
- **Alan bazlı (domain-driven)** mimarilerde bileşenler alanlara göre gruplanır.
- Katmanlı yapıda örneğin “müşteri” alanı her katmanda dağılmış olabilir.
- Bu durum değişiklik yapmayı zorlaştırabilir.
- **Sonuç:** Katmanlı mimari DDD yaklaşımıyla iyi örtüşmez.



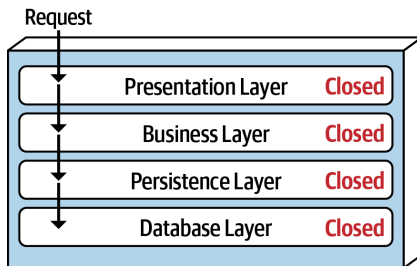
Katmanlı Mimari: Fiziksel Topoloji Varyantları

- Katmanlar mantıksal olarak ayrı olsa da fiziksel dağıtım değişebilir:
 - **Varyant 1:** Tüm katmanlar + veri tabanı tek dağıtım biriminde (gömülü veritabanı, küçük uygulamalar)
 - **Varyant 2:** UI ve veritabanı ayrı, geri kalan katmanlar birlikte
 - **Varyant 3:** Tüm katmanlar tek dağıtım biriminde (monolit)
- Gerçek hayatta çoğu **on-prem** uygulama varyant 3'e göre hazırlanmıştır.



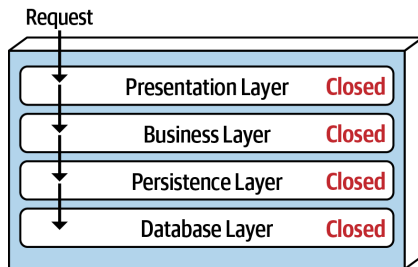
Yalıtımlı Katmanlar (Layers of Isolation)

- Her katman yalnızca bir alttaki katmanla iletişim kurmalı (Closed Layer).
- Amaç: Değişikliklerin sadece kendi katmanını etkilemesi.
- Açık katmanlar (Open Layer), performans için kısa yol sunsa da izolasyonu bozar.
- İzolasyon sayesinde sistem:
 - Daha az bağımlı,
 - Daha kolay test edilebilir,
 - Daha esnek ve sürdürülebilir olur.



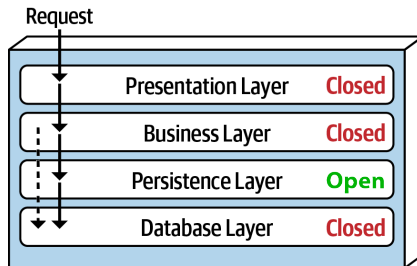
Kapalı Katmanlar (Closed Layers)

- Her katman yalnızca doğrudan altındaki katmanla iletişim kurar.
- Katmanlar arası sıkı izolasyon sağlar.
- Değişiklikler diğer katmanları etkilemeden yapılabilir.
- Yaygın mimari örneği: UI → İş Katmanı → Veri Katmanı → Veritabanı
- Avantajlar:
 - Gevşek bağlılık (loose coupling)
 - Daha kolay bakım ve test
 - Katmanlar bağımsız geliştirilebilir



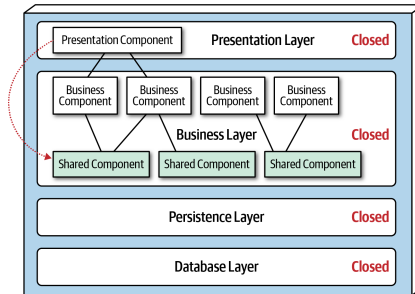
Açık Katmanlar (Open Layers)

- Katmanlar, alt katmanları atlayarak daha alt katmanlara erişebilir.
- Örnek: UI katmanının doğrudan veritabanına bağlanması.
- Bu yapı, performans için cazip olabilir (örn. Fast-Lane Reader).
- Ancak:
 - İzolasyon bozulur,
 - Katmanlar arası bağımlılık artar,
 - Sistem kırılgan hale gelir.
- Değişiklikler, birden fazla katmanı etkileyebilir.



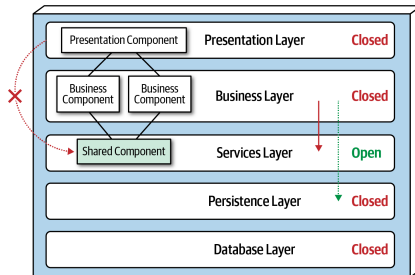
Katman Ekleme

- Sunum katmanının erişmesini istemediğimiz bazı paylaşılan bileşenler iş katmanında yer alabilir:
 - Örneğin günlükleme ve denetim bileşenleri (AuditLogger)
- Ancak sunum katmanı, iş katmanına erişebildiği için bu bileşenlere de erişebilir.
- Bu durum, katmanlar arası bağımlılığı artırır.



Katman Ekleme: Servis Katmanı ile Çözüm

- Paylaşılan bileşenler ayrı bir **servis katmanına** taşınır.
- Servis katmanı **açık (open)** katman olarak tanımlanır:
 - Sunum katmanı yalnızca iş katmanına erişebilir.
 - İş katmanı isterse bu servis katmanını kullanabilir.
- Böylece:
 - Sunum katmanı gereksiz bağımlılıklara sahip olmaz,
 - Katman izolasyonu korunur,
 - Mimaride net ayırım ve sürdürülebilirlik sağlanır.



Neden Katmanlı Mimari Tercih Edilir?

- Küçük ve basit uygulamalar için uygun bir mimari stildir.
- Bütçe ve zaman kısıtları olan projelerde hızlı başlangıç sağlar.
- Yaygın olarak bilindiği için geliştirici ekipler arasında öğrenme ve geliştirme maliyeti düşüktür.
- İş ihtiyaçları ve mimari tercihler henüz netleşmemişse başlangıç noktası olarak kullanılabilir.
- Ancak uygulama büyüdükçe:
 - Sürdürülebilirlik,
 - Esneklik,
 - Test edilebilirlik,
 - Dağıtım kolaylığı gibi karakteristikler zayıflar.
- Büyük ve karmaşık sistemler için daha modüler mimariler tercih edilmelidir.

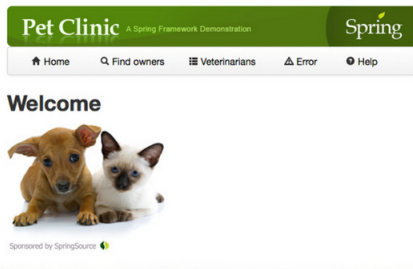


Spring PetClinic: Katmanlı Mimari Örneği

- Spring PetClinic, Spring Framework'ü ile hazırlanmış, **katmanlı mimari** stiline göre tasarlanmış bir örnek uygulamadır.

Github reposu

<https://github.com/spring-projects/spring-petclinic>



Katmanlar Arası Ayrım: PetClinic Örneği

- Uygulama genel olarak şu katmanlara ayrılmıştır:
 - **Controller:** HTTP isteklerini yöneten REST/Web Controller'lar (OwnerController, PetController).
 - **Service:** İş mantığı (Business Logic) katmanı.
 - **Repository:** Spring Data JPA ile veritabanı işlemleri (OwnerRepository, PetRepository).
 - **Model:** Owner, Pet, Vet, Visit gibi JPA Entity'leri.
- Her katman yalnızca kendisinden bir alt katmana erişim sağlar (Closed Layer yaklaşımı).
- Bu yapı, **ayrık sorumluluk**, **test edilebilirlik** ve **bakım kolaylığı** sağlar.



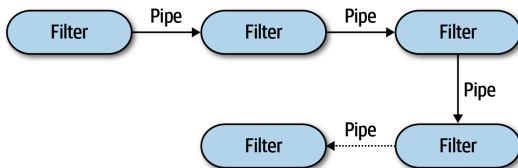
Genel Skor

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★



Boruhattı (Pipeline) Mimari Stili

- Katmanlı mimariden sonra gelen temel stillerden biridir.
- Unix kabuk (shell) dillerinde (Bash, Zsh) ve MapReduce modellerinde sıkça görülür.
- İşlevsellik bağımsız parçalara bölündüğünde doğal olarak ortaya çıkan bir yapı.
- Geliştiricilere modülerlik ve yeniden kullanım imkânı sunar.
- Fonksiyonel programlamadaki *map*, *reduce* gibi işlemlerle benzerlik gösterir.



Boruhattı Mimarisi: Topoloji

- **Pipe (Boru):** Filtreler arası veri taşıyan, tek yönlü bağlantılardır.
- **Filter (Filtre):** Her biri tek bir görevi üstlenen bağımsız ve durumsuz bileşenlerdir.
- Dört filtre türü vardır:
 - **Producer:** Süreci başlatır (veri üretir).
 - **Transformer:** Veriyi dönüştürür (map benzeri).
 - **Tester:** Veriyi test eder, süzgeç uygular (reduce benzeri).
 - **Consumer:** Süreci sonlandırır (veriyi kaydeder veya gösterir).
- Her filtre, küçük ve bağımsız görevlerle sistemin geneline katkı sağlar.



Bir Pipeline Örneği: Apache NiFi Nedir?

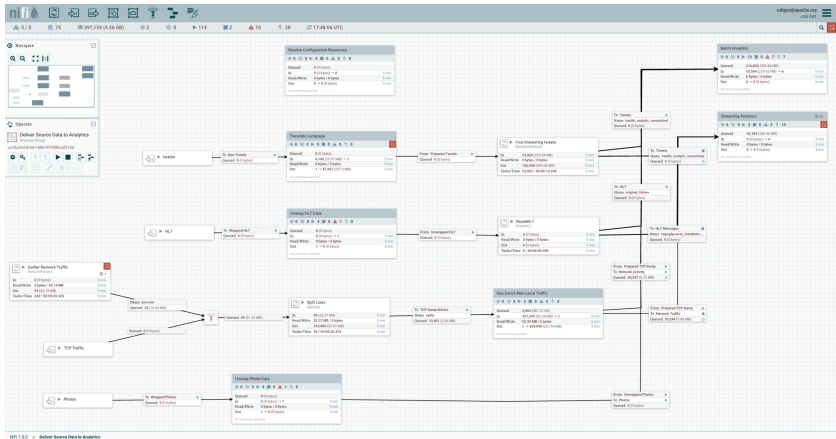
- Apache NiFi, veri akışlarını tasarlamak ve yönetmek için geliştirilmiş bir **veri yönlendirme ve dönüştürme platformudur**.
- **Pipeline (boruhattı) mimari stilini** temel alır.
- Her bir adım (işlem) bağımsız bir **“processor”** bileşeni olarak modellenir ve bu bileşenler veri akışlarıyla bağlanır.
- Kullanıcı dostu bir grafiksel arayüz sunar.

Github reposu

<https://github.com/apache/nifi>



Bir Pipeline Örneği: Apache NiFi Nedir?



Apache NiFi Nasıl Çalışır?

- Veri akışı şu bileşenlerle modellenir:
 - **Processor:** Her biri veri üzerinde bir işlem (okuma, dönüştürme, filtreleme vb.) yapar.
 - **Connection:** İşlem adımlarını birbirine bağlayan veri yollarıdır (pipe).
 - **FlowFile:** İşlenen veri paketidir; her işlem adımında değişebilir.
- Her işlem tek bir işi yapar — **stateless ve bağımsızdır** (filter).
- Bir veri kaynağından başlayıp adım adım dönüşümle hedefe ulaşır (pipeline akışı).



Apache NiFi Ne İçin Kullanılır?

- **Gerçek zamanlı veri işleme ve yönlendirme** ihtiyacı olan sistemlerde kullanılır.
- Yaygın kullanım alanları:
 - Log toplama ve yönlendirme
 - Sensör verisi işleme (IoT)
 - Veri göçü ve entegrasyonu
 - Veritabanları ve bulut sistemleri arası veri transferi
- Kurumsal ortamlarda **görsel olarak izlenebilir ve kontrol edilebilir** veri boru hatları kurmak için idealdir.



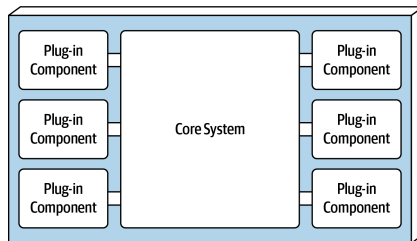
Genel Skor

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★ ★
Elasticity	★
Evolutionary	★ ★ ★
Fault tolerance	★
Modularity	★ ★ ★
Overall cost	★ ★ ★ ★ ★
Performance	★ ★
Reliability	★ ★ ★
Scalability	★
Simplicity	★ ★ ★ ★ ★
Testability	★ ★ ★



Mikroçekirdek (Microkernel) Mimarisi

- Diğer adıyla **plug-in (eklenti)** mimarisi olarak da bilinir.
- Temelinde bir **çekirdek sistem (core system)** ve bu çekirdeği genişleten bağımsız **plug-in (eklentiler)** yer alır.
- Başlangıçta **ürün-tabanlı yazılımlar** için geliştirilmiştir (ör. Eclipse IDE), ancak özel iş uygulamalarında da yaygın şekilde kullanılır.
- **Esneklik, genişletilebilirlik ve özelleştirilebilirlik** ihtiyaçlarına uygun bir mimari stildir.



Mikroçekirdek Mimarisinde Topoloji

- Yapı iki ana bileşenden oluşur:
 - **Core System (Çekirdek):** Uygulamanın temel işlevlerini barındırır (ör. eklenti yükleyici, servis sağlayıcı).
 - **Plug-in Bileşenleri:** Core üzerine eklenen bağımsız modüllerdir; işlevselliği genişletir.
- Plug-in bileşenleri, çekirdek ile kontrollü bir şekilde iletişim kurar.
- Bu yapı sayesinde uygulamalar:
 - Farklı senaryolara kolayca uyarlanabilir,
 - Yeni işlevler eklenebilir veya kaldırılabilir,
 - Daha düşük bağımlılıkla geliştirilebilir.

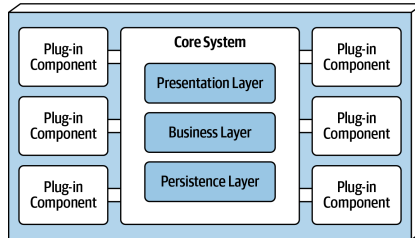


Core System (Çekirdek Sistem) Nedir?

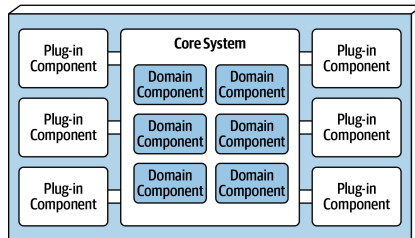
- Uygulamanın çalışması için gerekli olan **minimum işlevselliği** barındırır.
- **Karar verme, yönlendirme, eklenti yükleme** gibi altyapı görevlerini yerine getirir.
- Özel mantıklar ve genişletilmiş işlevler çekirdek dışında tutulur.
- Bu ayrım sayesinde:
 - **Genişletilebilirlik** ve **test edilebilirlik** artar.
 - **Bakım kolaylığı** sağlanır.
 - **Çevik geliştirme** desteklenmiş olur.
- Çekirdek sistem ister katmanlı mimariyle, ister DDD ile uygulanabilir.



Core System (Çekirdek Sistem) Nedir?



Layered Core System (Technically Partitioned)

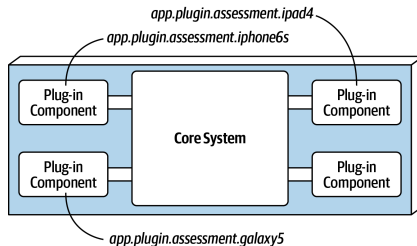


Modular Core System (Domain Partitioned)



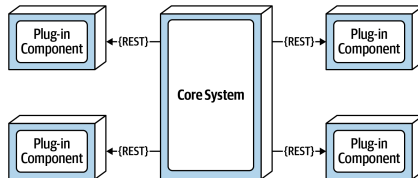
Eklenti Bileşenleri (Plug-In Components)

- Core sistemi genişleten, özelleştiren bağımsız bileşenlerdir.
- Genellikle değişkenliği yüksek kodlar burada yer alır:
 - Bakımı ve test edilebilirliği kolaylaştırır.
- Plug-in bileşenleri birbirinden bağımsız olmalı, aralarında doğrudan bağımlılık olmamalıdır.



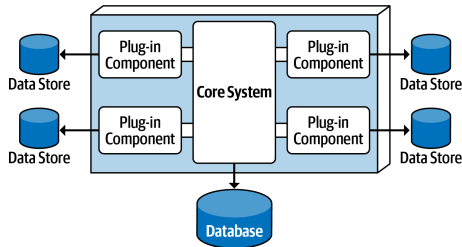
Eklenti Bileşenleri (Plug-In Components)

- Core ile iletişim genellikle noktadan noktaya şeklindedir:
 - Metot çağırısı, sınıf yükleme, fonksiyon çağırısı
 - JAR, DLL, namespace, package gibi yapılarla entegrasyon
- Alternatif olarak:
 - REST, mesajlaşma (messaging) gibi mekanizmalarla da çağrılabilir.
 - Ancak sistem hâlâ tek bir mimari stildir (çekirdek sistem monolittir).



Eklenti Bileşenleri (Plug-In Components)

- Ortak veritabanına doğrudan erişim önerilmez; veri core üzerinden geçirilmelidir.
- Veya her eklentinin kendi veritabanı olabilir.



Registry (Kayıt Sistemi)

Registry, çekirdek sistemin hangi *plug-in* bileşenlerinin mevcut olduğunu bilmesini ve onlara erişmesini sağlayan yapıdır.

- Her *plug-in* bileşeni hakkında bilgi içerir:
 - Adı
 - Veri sözleşmesi (data contract)
 - Bağlantı türü: Noktadan noktaya, mesajlaşma, REST vb.
- Genellikle bir **map/dictionary** veri yapısı ile uygulanır.

Registry, çekirdeğin eklentileri dinamik olarak bulmasına ve çağırmasına olanak tanır.



Registry (Kayıt Sistemi)

Registry Örneği

```
Map<String, String> registry = new HashMap<>();
static {
    // Noktadan noktaya bağlantı
    registry.put("iPhone6s", "Iphone6sPlugin");

    // Mesajlaşma sistemi
    registry.put("iPhone6s", "iphone6s.queue");

    // REST API ile bağlantı
    registry.put("iPhone6s", "https://atlas:443/assess/iphone6s");
}
```



Kontratlar

Kontratlar, çekirdek sistem ile plug-in bileşenleri arasındaki anlaşmadır.

- Bu anlaşma, plug-in'in hangi davranışları (metotları) sunacağını, ne tür veri alıp vereceğini tanımlar.
- Böylece çekirdek sistem, plug-in'in iç işleyişini bilmeden onunla çalışabilir.

Neden Önemlidir?

- **Bağımsız geliştirme:** Farklı plug-in'ler, çekirdekten bağımsız geliştirilip entegre edilebilir.
- **Tutarlılık:** Plug-in'ler ortak bir arayüze (interface) uyarak sistem bütünlüğünü korur.
- **Uyarılama imkânı:** Üçüncü taraf plug-in'ler farklı konratlara sahipse, bir adapter ile uyum sağlanabilir.
- **Standart veri iletimi:** XML, JSON veya nesne tabanlı iletim mümkündür.



Kontrat Örneği

Kontrat Örneği

```
public interface AssessmentPlugin {  
    public AssessmentOutput assess();  
    public String register();  
    public String deregister();  
}  
  
public class AssessmentOutput {  
    public String assessmentReport;  
    public Boolean resell;  
    public Double value;  
    public Double resellPrice;  
}
```



Microçekirdek Mimari Stili Örneği: Eclipse IDE (1/2)

Eclipse Nedir?

- Java ile geliştirilmiş açık kaynaklı bir entegre geliştirme ortamıdır (IDE).
- Microkernel mimarisine uygun olarak bir **çekirdek sistem** ve **plug-in bileşenlerden** oluşur.
- Zengin işlevselliği yüzlerce bağımsız eklenti (plug-in) ile sunar.

Microkernel Mimarının Eclipse'teki Yansıması

- **Çekirdek Sistem:** Uygulama başlatma, temel UI altyapısı, yaşam döngüsü yönetimi.
- **Plug-in'ler:** Java editörü, Git entegrasyonu, C++, Python, PHP desteği vb.
- **Üçüncü parti plug-in desteği:** Docker, Spring Boot, Kubernetes gibi araçlar için genişleme imkânı.



Microkernel Mimari Stili Örneği: Eclipse IDE (2/2)

Plug-in Mekanizması

- Her eklenti bir **OSGi bundle** olarak yapılandırılır.
- Plug-in'ler bağımsızdır ve yalnızca çekirdek sistemle kontrat üzerinden iletişim kurar.
- `plugin.xml` ve `MANIFEST.MF` dosyaları ile yapılandırılır.

Eklenti Yükleme Süreci

- 1 Kullanıcı yeni bir özellik talep eder (örnek: Java desteği).
- 2 Eclipse, ilgili plug-in'i yükler.
- 3 Plug-in çekirdeğe `register()` metodu ile kaydolur.
- 4 Plug-in çalışmaya başlar.

Avantajları

- Yüksek genişletilebilirlik ve özelleştirilebilirlik.
- Bağımsız geliştirme ve test imkânı.
- Çekirdek sisteme dokunmadan yeni özellik eklenebilir.



Genel Skor

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★ ★ ★
Elasticity	★
Evolutionary	★ ★ ★
Fault tolerance	★
Modularity	★ ★ ★
Overall cost	★ ★ ★ ★ ★
Performance	★ ★ ★
Reliability	★ ★ ★
Scalability	★
Simplicity	★ ★ ★ ★
Testability	★ ★ ★

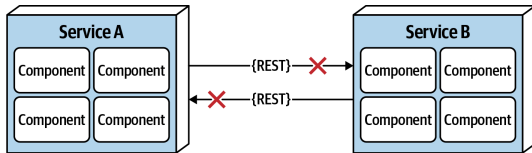


Dağıtık Mimari Stiller

- Dağıtık mimari stiller:
 - Servis tabanlı mimari
 - Olay tabanlı mimari
 - Alan tabanlı mimari
 - Servis yönelimli mimari
 - Mikroservis mimarisi
- Dağıtık mimari stiller, monolitik mimari tarzlarına göre performans, ölçeklenebilirlik ve kullanılabilirlik açısından çok daha güçlüdür.
- Bu güç için önemli trade-off'lara sahiptir.
- İlk olarak 1994 yılında L. Peter Deutsch ve Sun Microsystems'den diğer meslektaşları dağıtık stillere dair yanılgılar tanımlamıştır.

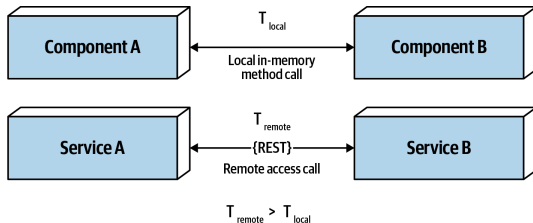


Yanılğı 1 - Ağ güvenilirirdir



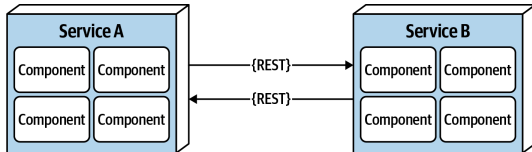
- Ağ güvenilir değildir.
- Teknoloji geliştikçe bu dezavantajın etkisi azaltılmaya çalışılsa da tamamen güvenilir bir ağ mümkün değildir.

Yanılğı 2 - Gecikme sıfırdır



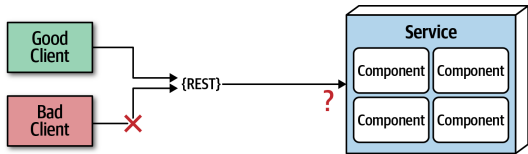
- Gecikme hiçbir zaman sıfır olamaz.
- Lokal bileşenler arası iletişim her zaman daha hızlı olacaktır.

Yanılğı 3 - Bantgenişliği sonsuzdur



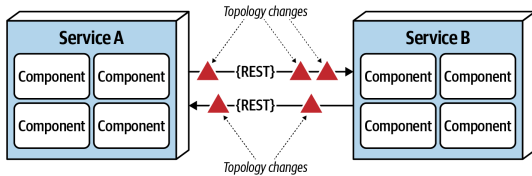
- Bantgenişliği sonsuz olamaz.
- Büyük veri transferleri ağı tıkayabilir veya maliyeti artırabilir.

Yanılğı 4 - Ağ güvenlidir



- Ağ iletişiminin varsayılan olarak güvenli olduğu düşünülür.
- Veri gizliliği, kimlik doğrulama ve yetkilendirme gibi güvenlik önlemleri şarttır.
- MITM (Man-in-the-Middle) saldırıları veya veri sızıntıları riski vardır.

Yanılğı 5 - Topoloji değışmez



- Ağdaki düğümelerin (nodes) sabit kalacağı varsayılır.
- Sunucular eklenebilir/kaldırılabilir, IP adresleri değışebilir.
- Sistemler dinamik topolojilere uyum sağlamalıdır.

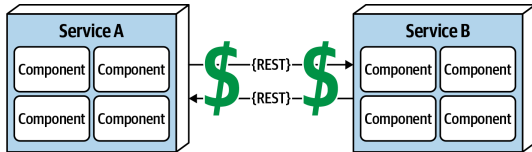
Yanılğı 6 - Topoloji değışmez



- Tüm sistemi yöneten tek bir otorite olduğu varsayılır.
- Dağıtık sistemlerde farklı bileşenler farklı ekipler tarafından yönetilir.
- Uyumlu çalışma için standartlar ve API sözleşmeleri gerekir.

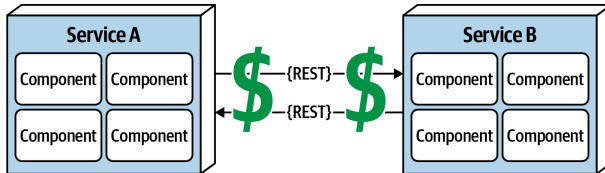


Yanılğı 7 - Taşıma maliyeti sıfırdır



- Veri taşımının maliyetsiz olduğu düşünülür.
- Veri transferi (özellikle bulut ortamlarında) maliyetlidir.
- Gereksiz veri transferinden kaçınmak için optimizasyon yapılmalıdır.

Yanılıgı 8 - Ağ homojendir



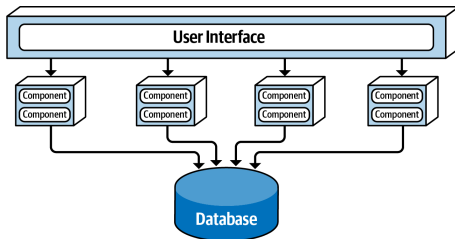
- Tüm ağ bileşenlerinin aynı protokolleri ve teknolojileri kullandığı varsayılır.
- Farklı cihazlar, işletim sistemleri veya protokoller (HTTP/gRPC/WebSocket) bir arada çalışabilmelidir.

Servis Temelli Mimari Stil (Service Based Architecture)

- Servis-temelli mimari stili, mikroservis mimarisinin bir tür melezi olarak kabul edilir.
- Dağıtık bir mimari stilidir, ancak mikroservis veya olay güdümlü mimariler kadar karmaşık ya da maliyetli değildir.
- Bu sayede özellikle iş uygulamaları için oldukça pragmatik ve popüler bir tercihtir.
- Esneklik ve uygulanabilirlik açısından avantajlıdır, bu nedenle birçok kurum tarafından yaygın olarak benimsenmektedir.



Servis-Temelli Mimari: Topoloji



- Servis-temelli mimarinin topolojisi, dağıtık bir yapıda ama **makro düzeyde katmanlı bir mimari** olarak tanımlanır.
- Temel bileşenler:
 - Dağıtık veya monolitik bir kullanıcı arayüzü (UI),
 - Ayrı dağıtılan, genellikle katmanlı veya alan odaklı servisler,
 - Ortak kullanılan tek bir monolitik veritabanı.



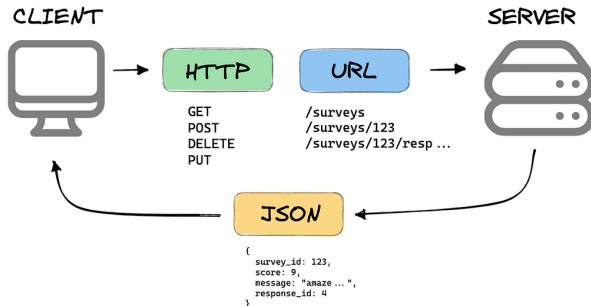
Servis-Temelli Mimari: Topoloji

- Bu mimari tarzındaki servisler tipik olarak bağımsız ve ayrı olarak konuşlandırılan **kaba taneli** “uygulama bölümleridir”.
- UI ile servisler arasında erişim genellikle **REST** ile olur; ancak Remote Procedure Call (RPC) veya mesajlaşma da kullanılabilir.
- Servisler çoğunlukla doğrudan UI tarafından erişilir; bazen API gateway veya proxy katmanı da eklenebilir.
- Paylaşılan veritabanı sayesinde servisler *SQL join* gibi geleneksel sorgularla çalışabilir.



REST

- REST (Representational State Transfer), bir mimari stildir; ancak bu, uygulama mimarisi değil, web servisleri veya istemci-sunucu iletişimi için kullanılan bir iletişim mimarisi stildir.
- Yani REST, katmanlı mimari, mikroservis veya servis-tabanlı mimari gibi bileşenlerin kendi içinde nasıl yapılandırıldığını değil, bu bileşenlerin birbiriyle nasıl konuştuğunu tanımlar.



REST'in temelleri

- ❶ **İstemci - Sunucu ayrımı:** İstemci (örneğin bir web tarayıcısı) ve sunucu (örneğin bir API) birbirinden bağımsızdır.
- ❷ **Durumsuzluk (Stateless):** Her istek bağımsızdır, önceki isteğin durumunu bilmez.
- ❸ **Kaynak temelli iletişim:** Tüm işlemler kaynaklar (örneğin kullanıcı/123) üzerinde yapılır.
- ❹ **HTTP yöntemlerini kullanır:**
 - GET : Veri alma
 - POST : Yeni veri oluşturma
 - PUT : Veriyi güncelleme
 - DELETE : Veriyi silme
- ❺ **Temsil (Representation):** Sunucudan dönen veri JSON, XML gibi bir biçimde gelir.



REST

- REST, nasıl bir iletişim kurulacağının prensiplerini belirler, iletişimin kendisi değildir.
- HTTP protokolü, REST prensiplerini uygulamak için bir araçtır.
- Diyagramlarda "haberleşme aracı" gibi gösterilmesinin nedeni, genellikle RESTful API'ların katmanlar arasında köprü olarak kullanılmasıdır.



Örnek

HTTP İstek

```
GET /pizzas HTTP/1.1  
Host: api.pizza.com  
Accept: application/json
```

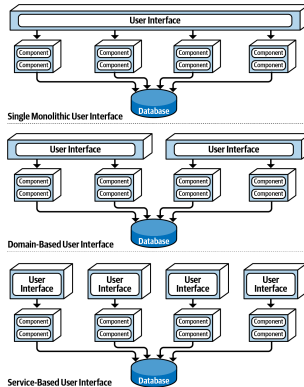
JSON Cevap

```
[  
  { "id": 1, "name": "Margherita", "price": 10.99 },  
  { "id": 2, "name": "Pepperoni", "price": 12.99 }  
]
```



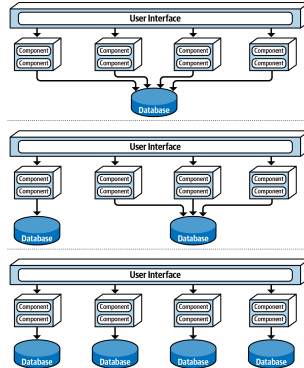
Servis Temelli Mimari Stil Topoloji Varyantları

- Monolitik UI duruma göre alan (domain) bazlı arayüzlere hatta servis bazlı arayüzlere bölünebilir.



Servis Temelli Mimari Stil Topoloji Varyantları

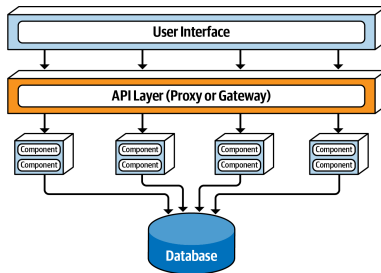
- Benzer şekilde, tek bir monolitik veritabanını ayrı veritabanlarına ayırma,
- hatta her bir etki alanı servisiyle eşleşen etki alanı kapsamındaki veritabanlarına (mikro servislere benzer) kadar gitme fırsatları olabilir.



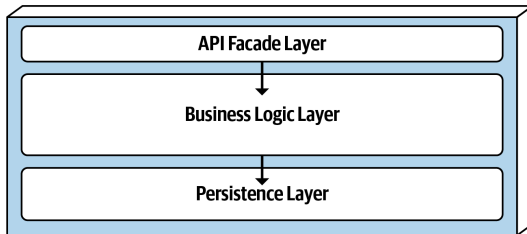
Servis Temelli Mimari Stil Topoloji Varyantları

UI ile servisler arasına eklenecek API katmanı:

- Güvenlik, logging, auditing gibi ortak işlemleri merkezi yapar.
- hUI ile servisleri doğrudan bağlamak yerine bir “köprü” görevi görür.
- Servislerin dış dünyaya kontrollü açılmasını sağlar.
- UI sade kalır; karmaşık altyapı işleri bu katmana devredilir.

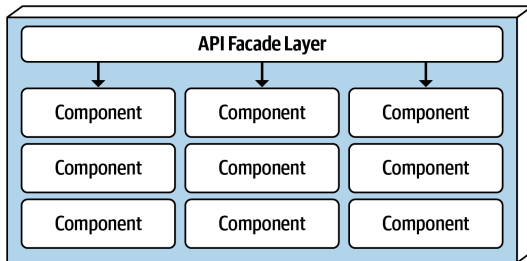


Servis Tasarımı



Layered design (technical partitioning)

Domain design (domain partitioning)



Katmanlı Servis Tasarımı

Servis-tabanlı mimaride domain servisleri genellikle katmanlı mimari stilde tasarlanır. Bu yapıda her servis şu katmanlardan oluşur:

- **API (Facade) Katmanı**

Servise gelen dış istekleri (REST, RPC vs.) karşılar.

- **Business (İş) Katmanı**

Uygulama mantığı ve iş kuralları burada yer alır. Örn: sipariş onayı, stok kontrolü.

- **Persistence (Veri) Katmanı**

Veritabanı işlemleri bu katmanda yapılır. DAO, repository gibi yapılar içerir.

Avantajları:

- Klasik ve tanıdık bir mimari yaklaşımdır.
- Geliştirmesi ve test etmesi görece kolaydır.



Alt-Domainlere Bölünmüş Tasarım

Alternatif olarak, bir domain servisi kendi içinde **alt-domain'lere bölünebilir**. Bu yaklaşım, *modüler monolit* stiline benzer.

Örnek: *Kullanıcı Servisi* → Kimlik yönetimi, adres yönetimi, kullanıcı ayarları gibi alt bölümler.

Her alt-domain:

- Kendi iş mantığı ve veri erişim katmanına sahiptir.
- Birbirinden bağımsız geliştirilebilir.

Avantajları:

- Modülerlik ve okunabilirlik artar.
- Farklı ekipler paralel çalışabilir.
- Mikroservise dönüşüm kolaylaşır.



Veritabanı Bölütleme (Database Partitioning) Nedir?

- Veritabanının mantıksal olarak domain alanlarına ayrılmasıdır.
- Her domain kendi verisini kullanır, böylece bağımsız hale gelir.
- Servislerin kendi veri alanlarına odaklanması sağlanır.
- Paylaşılan kütüphaneler (shared libraries) ile kod tekrarı azaltılır.

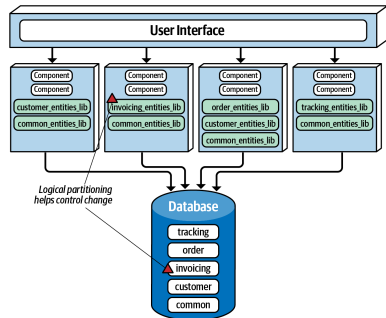
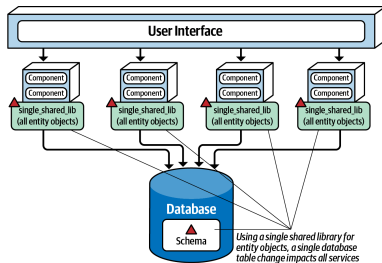


Neden Veritabanı Bölütlenir?

- **Değişiklik riski azaltılır:** Tablolarda yapılan değişiklikler yalnızca ilgili domain'i etkiler.
- **Bakım kolaylığı:** Her domain kendi içinde yönetilir.
- **Kod tekrarı azalır:** CRUD işlemleri ve modeller shared library olarak tekrar kullanılabilir.
- **Mikroservise geçiş kolaylaşır:** Her domain veri olarak ayrılmıştır.



Örnek: Mantıksal Veritabanı Bölütleme



Örnek: Mantıksal Veritabanı Bölütleme

Veritabanı Domain'leri:

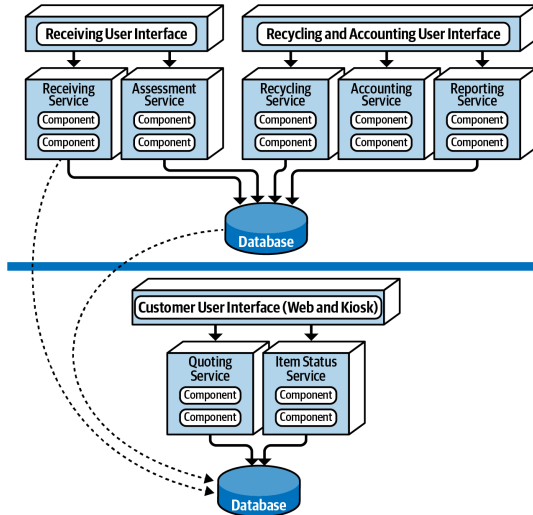
- common
- customer
- invoicing
- order
- tracking

Shared Libraries:

- common-lib.jar
- customer-lib.jar
- invoicing-lib.jar
- order-lib.jar
- tracking-lib.jar



Elektronik Geri Dönüşüm Sistemi (Topoloji)



Servis Tabanlı Mimari Örneği

- Elektronik geri dönüşüm şirketi, müşterilerin eski cihazlarını teslim edip ücret alabileceği bir sistem sunar.
- İş akışı yedi temel adıma ayrılır ve her adım bir domain servisine dönüştürülür.
- Servisler bağımsız olarak dağıtılmıştır.



Domain Servisleri

- **Quoting:** Cihaz için teklif alma
- **Receiving:** Cihazın fiziksel olarak alınması
- **Assessment:** Cihaz durumunun değerlendirilmesi
- **Accounting:** Ödeme işlemleri
- **Item Status:** İşlem durumu sorgulama
- **Recycling:** İmha veya satış işlemi
- **Reporting:** İç raporlama



Servislerin Ölçeklenebilirliği ve Dağıtımı

- Her domain servisi ayrı ayrı dağıtılmıştır.
- **Quoting** ve **Item Status** servisleri daha fazla trafiğe maruz kaldığı için yatayda ölçeklenebilir.
- Diğer servisler genellikle tek örnek (instance) olarak yeterlidir.



Arayüz (UI) Ayrımı

- UI bileşenleri, ilgili domain servislerine göre gruplanmıştır.
- **Customer-facing, Receiving, Recycling and Accounting** şeklinde ayrılmıştır.
- Bu yapı güvenlik, hataya dayanıklılık ve daha iyi kullanıcı deneyimi sağlar.



Veritabanı Ayrımı ve Güvenlik

- Müşteriyle ilgili işlemler ve iç operasyonlar için ayrı veritabanları kullanılır.
- Dahili servisler, müşteri veritabanına erişebilir; dışarıdan erişim kapalıdır.
- Gerekirse tablo yansıtma ve senkronizasyon gibi yöntemlerle veri paylaşımı sağlanabilir.



Sonuçlar ve Mimari Değerlendirme

- Servis tabanlı mimari, hem dağıtık sistem esnekliğini hem de uygulanabilir sadeliği sunar.
- Güvenlik, ölçeklenebilirlik ve domain bazlı ayırım sayesinde sürdürülebilir bir yapı sağlanır.



Genel Skor

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★



Olay Tabanlı Mimari Nedir?

- **Event-Driven Architecture (EDA)**, dağıtık ve asenkron yapısıyla yüksek performans ve ölçeklenebilirlik sunan bir mimari stildir. Küçük ve büyük ölçekli sistemlerde rahatlıkla kullanılabilir.
- Bu mimaride, bileşenler birbirinden bağımsız çalışır ve sadece gerçekleşen **olaylara tepki verir**.
- EDA, tek başına bir mimari stil olarak uygulanabilir veya mikroservis gibi diğer mimari stillerin içinde kullanılabilir.

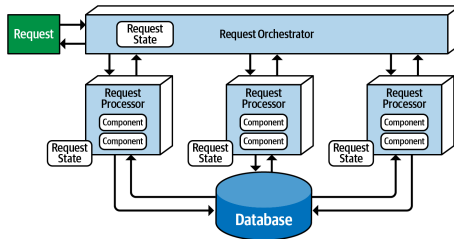


İstek Tabanlı ve Olay Tabanlı Modeller

Genelde istek tabanlı modeller ve olay tabanlı modeller arasında bir nüans vardır.

- **İstek Tabanlı Model (Request-Based):**

- Kullanıcıdan gelen isteği belirli işlemciler işler.
- İş akışı: Arayüz/API → Orkestratör → İşlemciler → Veritabanı.
- *Örnek:* Sipariş geçmiş sorgulama.



İstek Tabanlı ve Olay Tabanlı Modeller

Genelde istek tabanlı modeller ve olay tabanlı modeller arasında bir nüans vardır.

- **Olay Tabanlı Model (Event-Based):**

- Sisteme bir olay gerçekleştiğinde otomatik olarak tepki verilir.
- *Örnek:* Açık artırmada teklif verme olayı.
- Bileşenler, tetiklenen olaya göre harekete geçer.



Olay Tabanlı Mimari: Topolojiler

Olay tabanlı mimaride iki temel topoloji bulunur:

- **Mediator (Yönetici) Topolojisi:** Olay işleme akışı üzerinde merkezi kontrol gerektiğinde tercih edilir.
- **Broker (Aracı) Topolojisi:** Yüksek tepki hızı ve esnek işlem kontrolü gerektiğinde kullanılır.

Bu iki yapı, farklı mimari ihtiyaçlara uygun olacak şekilde tasarlanmıştır.



Broker Topolojisi (Olay Tabanlı Mimari)

Broker topolojisi, merkezi bir yönlendirici olmadan çalışan dağıtık bir yapıdır. Olaylar, bir aracı (broker) üzerinden zincirleme şekilde işlenir.

Temel bileşenler:

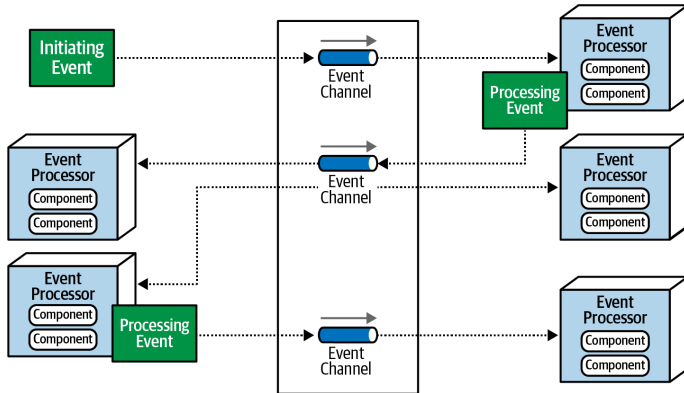
- **Başlatıcı Olay:** Süreci tetikleyen ilk olay.
- **Olay Aracısı (Broker):** Olayları yönlendirir.
- **Olay İşleyici:** Olayı işler ve yeni bir olay üretir.
- **İşlenmiş Olay:** Diğer bileşenleri tetikler.

Avantaj: Basit, esnek ve yüksek tepki süresi sağlar.

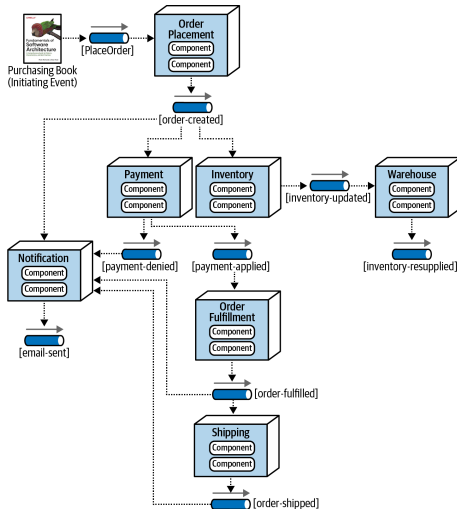
Mesajlaşma modeli: *Yayınla-abone ol (publish-subscribe)* modeli kullanılır.



Broker Topolojisi (Olay Tabanlı Mimari)



Broker Topolojisi (Olay Tabanlı Mimari)

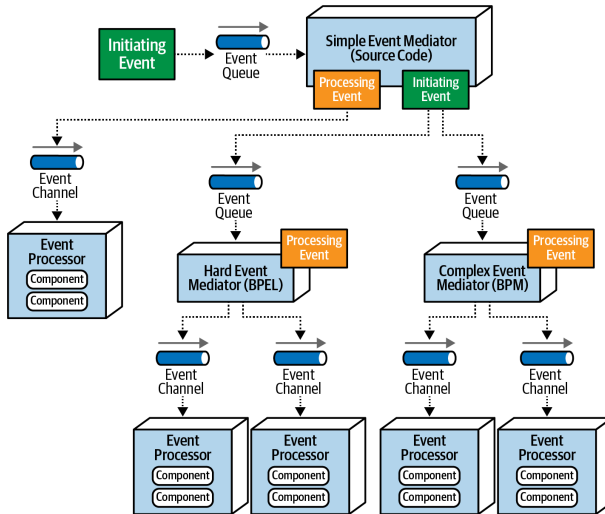


Mediator Topolojisi (Olay Tabanlı Mimari)

- **Mediator topolojisi**, olayların merkezi bir yapı (olay yöneticisi) tarafından kontrol edildiği bir modeldir.
- Amaç: Birden fazla olay işlemcisinin koordinasyonunu sağlamak.
- **Temel bileşenler:**
 - Başlatıcı olay (initiating event)
 - Olay kuyruğu
 - Olay yöneticisi (mediator)
 - Olay kanalları
 - Olay işlemcileri
- Olay yöneticisi, iş akışını bilir ve hangi işlemcinin ne yapacağını belirler.



Mediator Topolojisi (Olay Tabanlı Mimari)

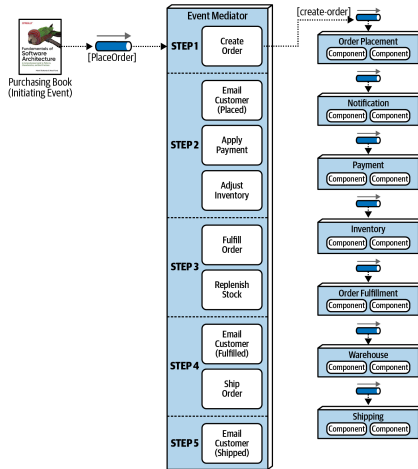


Mediator Topolojisi (Olay Tabanlı Mimari)

- 1 Başlatıcı olay, olay kuyruğuna gönderilir.
- 2 Olay yöneticisi (mediator) bu kuyruğu dinler ve iş akışını başlatır.
- 3 İlgili olay işlemcilerine, özel olay kanalları üzerinden görev gönderilir.
- 4 İşlemciler işleri tamamlar, ama sistemin geri kalanına duyuru yapmaz (broker'daki gibi “reklam” yoktur).
- 5 İhtiyaca göre her alan (örneğin müşteri, sipariş) kendi mediator'üne sahip olabilir.



Mediator Topolojisi (Olay Tabanlı Mimari)

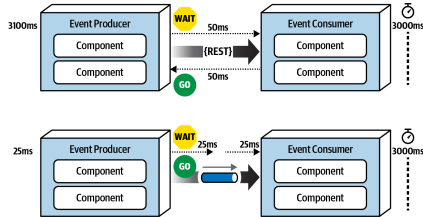


Asenkron Yetenekler: Olay Tabanlı Mimari

- Olay tabanlı mimari, işlemleri **tamamen asenkron** olarak yönetir.
- **Avantajları:**
 - Kullanıcı arayüzü daha hızlı yanıt verir.
 - Sistem bileşenleri gevşek bağlı olur, daha dayanıklıdır.
 - Yük artışında kuyruk yapısıyla esneklik sağlanır.

Örnek: Sipariş sistemi

- *Senkron*: Sipariş → Kart Çek → E-posta Gönder (hepsi sırayla)
- *Asenkron*: Sipariş alındı → OrderCreated olayı → ayrı servisler devreye girer



Asenkron Hata Yönetimi: Workflow Event Pattern

Problem: Asenkron iletişimde hata anında işlem durmaz, çağıran bileşen hata almaz.

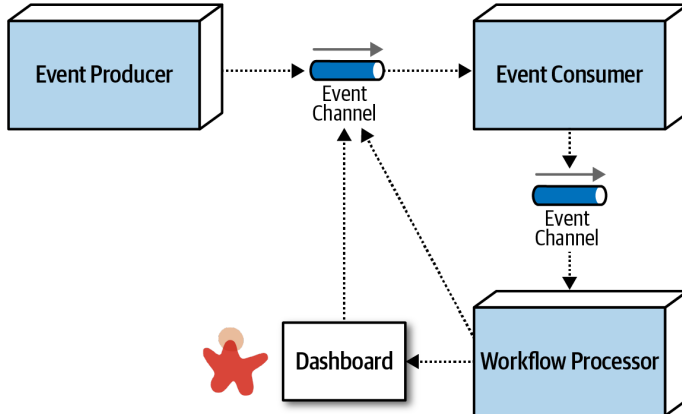
Çözüm: *Workflow Event Pattern* ile hatalar kontrol altına alınır.

- **Delegation (Yetki devri):** Hata yönetimi, özel bir *workflow delegate* bileşenine devredilir.
- **Containment (İzolasyon):** Hatalar yayılmadan sınırlı alanda tutulur; diğer olaylar etkilenmez.
- **Repair (Onarım):** Başarısız adımlar yeniden denenebilir veya manuel müdahaleye açık hale gelir.

Kazanç: Sistem, hem *resilient* (dayanıklı) hem de *responsive* (hızlı tepki veren) kalır.



Asenkron Hata Yönetimi: Workflow Event Pattern



Broadcast (Yayın) Kabiliyeti

Tanım: Event-driven mimaride bir olay (event), üretici tarafından yayınlandığında, aynı mesaj birden fazla tüketici (subscriber) tarafından alınabilir.

- Yayıncı, alıcıların kim olduğunu veya nasıl tepki vereceğini bilmek zorunda değildir.
- **Loose coupling (gevşek bağlılık)** sağlar.
- **Ölçeklenebilirlik** ve **genişletilebilirlik** açısından avantajlıdır.

Örnek: Bir ürün fiyatı değiştiğinde bu bilgi hem stok yönetimi hem de bildirim servislerine aynı anda ulaşabilir.



Genel Skor

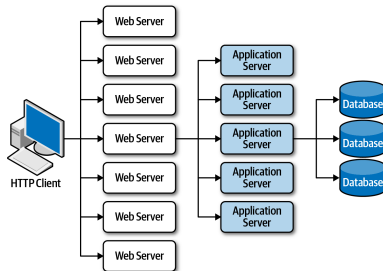
Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★ ★ ★
Elasticity	★ ★ ★
Evolutionary	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Modularity	★ ★ ★ ★
Overall cost	★ ★ ★
Performance	★ ★ ★ ★ ★
Reliability	★ ★ ★
Scalability	★ ★ ★ ★ ★
Simplicity	★
Testability	★ ★



Alan Tabanlı Mimari Stili: Giriş

Problem:

- Geleneksel web uygulamaları genellikle şu sırayı izler: Tarayıcı → Web Sunucusu → Uygulama Sunucusu → Veritabanı.
- Yük arttıkça darboğaz önce web sunucusunda, sonra uygulama sunucusunda, en sonunda da veritabanında oluşur.
- Veritabanını ölçeklendirmek maliyetli ve karmaşıktır — bu da sistemi en zayıf noktası hâline getirir.



Alan Tabanlı Mimari Stili: Giriş

Çözüm: Alan Tabanlı(Space-Based) Mimari Stili

- Değişken ve öngörülemeyen eşzamanlı kullanıcı yükleri için tasarlanmıştır.
- Yüksek ölçeklenebilirlik ve esneklik sağlar.
- Cache tabanlı çözümlerin sınırlarını aşar; veritabanı bağımlılığını azaltır.
- Performans darboğazlarını mimari düzeyde ortadan kaldırır.



Alan Tabanlı Mimari: Genel Topoloji

Temel Yaklaşım:

- Adını, paralel işlemcilerin ortak hafıza (tuple space) üzerinden iletişim kurduğu yaklaşımdan alır.
- Merkezi veritabanı senkron bağımlılık olmaktan çıkarılır. Bunun yerine, **replike edilmiş bellek içi veri gridleri (in-memory data grid)** kullanılır.

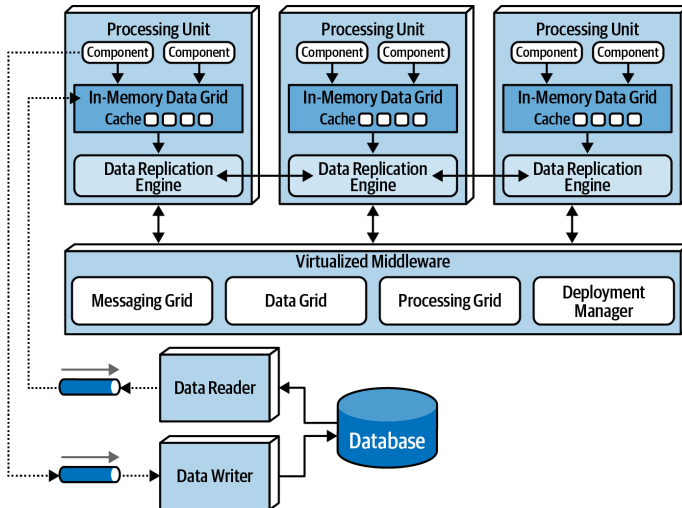
Temel Bileşenler:

- **İşlem Birimi (İB):** Uygulama kodunu çalıştırır; veriyi bellek içinde tutar.
- **Sanal Ara Katman:** İB'leri koordine eder ve yönetir.
- **Veri Pompası:** Veritabanına veri göndermek için güncellemeleri kuyruklar üzerinden iletir.
- **Veri Yazıcı/Okuyucu:** Veritabanına yazma ve başlangıçta veriyi İB'ye okuma işini üstlenir.

Avantajlar: Yük arttıkça İB'ler dinamik olarak devreye girer; kullanıcı yükü azalınca kapanır.

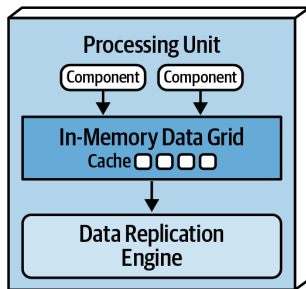


Alan Tabanlı Mimari: Genel Topoloji



İşlem Birimi (Processing Unit)

- Alan tabanlı mimarideki temel yürütücü bileşendir.
- Uygulama mantığını içerir (hem ön yüz hem de iş mantığı olabilir).
- Bellek içi verilerle çalışır; yüksek performans sağlar.



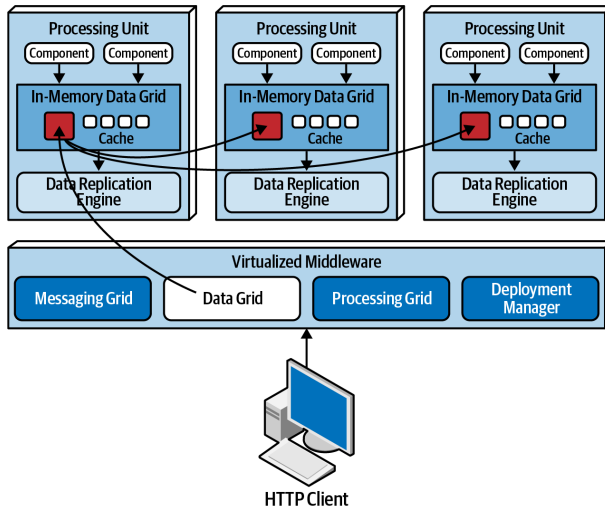
Sanal Ara Katman Bileşenleri (Virtualized Middleware)

Sanal ara katman, işlem birimleri arasında veri eşitleme ve istek yönlendirme altyapısını sağlar. Alt bileşenleri:

- **Mesajlaşma İzgarası (Messaging Grid):** İstekleri uygun işlem birimine yönlendirir. Genellikle yük dengeleyici (ör. Nginx, HAProxy) olarak çalışır.
- **Veri İzgarası (Data Grid):** İşlem birimleri arasında eşzamanlı olmayan (asenkron) veri replikasyonu sağlar. Her işlem birimi aynı bellek içi veriye sahiptir.
- **İşlem İzgarası (Processing Grid):** Birden fazla işlem biriminin koordine edilmesini gerektiren iş akışlarında aracılık yapar.
- **Dağıtım Yöneticisi (Deployment Manager):** Yük durumuna göre işlem birimlerini dinamik olarak başlatır veya kapatır; elastikliği sağlar.



Sanal Ara Katman Bileşenleri (Virtualized Middleware)



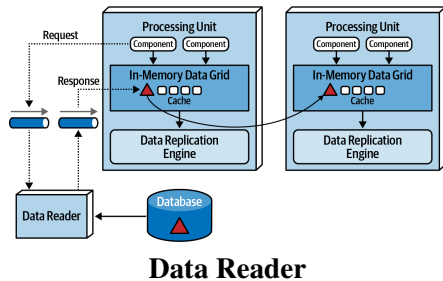
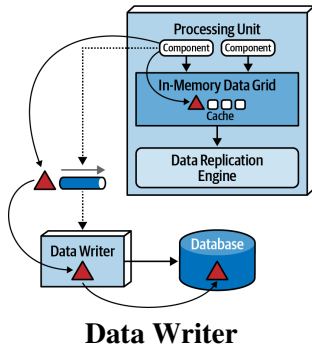
Veri Pompası ve Veri Yazıcı/Okuyucular

Veri akışının veritabanı ile entegrasyonu şu bileşenlerle sağlanır:

- **Veri Pompası (Data Pump):** Veritabanına yazılacak güncellenmiş verileri mesajlaşma altyapısı ile veri yazıcısına iletir.
- **Veri Yazıcısı (Data Writer):** Veri pompasından gelen mesajları alır ve veritabanına yazar.
- **Veri Okuyucusu (Data Reader):** Özel durumlarda (çökme, yeniden dağıtım, arşiv sorgusu) veriyi veritabanından okur ve işlem birimlerine gönderir.



Veri Pompası ve Veri Yazıcı/Okuyucular



Alan Tabanlı Mimari Örneği: Konser Bileti Satış Sistemi

- Düşük kullanıcı yükü → satış öncesi dönemde
- Satış açıldığında → ani ve yoğun trafik (binlerce eşzamanlı kullanıcı)
- Kullanıcılar iyi koltuklar için yarışır, veriler hızla güncellenmelidir
- Geleneksel veritabanı tabanlı mimari bu yükü kaldıramaz



Neden Alan Tabanlı Mimari?

- Ani yük artışlarını karşılayabilecek yüksek esneklik (elasticity)
- Veritabanı yerine bellek içi veri replikasyonu
- İşlem birimleri dinamik olarak başlatılıp durdurulabilir
- Kullanıcı deneyimi kesintisiz ve hızlı kalır



Veri Güncellemeleri Nasıl Yönetilir?

- Kullanıcı işlem yapar → işlem birimi bellekteki veriyi günceller
- Veri eşzamanlı olarak değil, **asen kron** şekilde veri pompasına iletilir
- Veri yazıcı (data writer) → kuyruğu dinler → veritabanına yazar
- Veritabanı üzerindeki yük zamana yayılır, sıkışma önlenir



Avantajlar

- Yüksek performans ve düşük gecikme
- Esnek ölçeklenebilirlik: yük artınca işlem birimi sayısı artar
- Veritabanı bağımlılığı azaltılır, sadece kalıcılık için kullanılır
- Gerçek zamanlı veri işleme bellekte çözülür



Genel Skor

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1 to many
Deployability	★ ★ ★
Elasticity	★ ★ ★ ★ ★
Evolutionary	★ ★ ★
Fault tolerance	★ ★ ★
Modularity	★ ★ ★
Overall cost	★ ★
Performance	★ ★ ★ ★ ★
Reliability	★ ★ ★ ★
Scalability	★ ★ ★ ★ ★
Simplicity	★
Testability	★



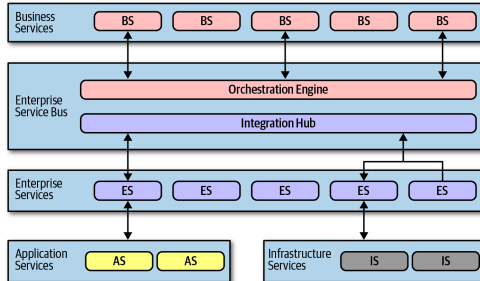
Orkestrasyon-Odaklı Servis-Yönelimli Mimari (OD-SOA): Temel Felsefe

- Yazılımı yeniden yazma ihtiyacını azaltmak amacıyla geliştirilmiştir.
- Kurumsal ölçekte servislerin yeniden kullanılabilirliğine odaklanır.
- Amaç: Tekrarlanan işlevleri merkezi hale getirerek sürdürülebilir mimari kurmak.



Katmanlı Yapı

- OD-SOA dört ana servis katmanından oluşur:
 - İş (Business) Servisleri
 - Kurumsal (Enterprise) Servisler
 - Uygulama (Application) Servisleri
 - Altyapı (Infrastructure) Servisleri
- Bu katmanlar orkestrasyon motoru ile entegre çalışır.



İş (Business) Servisleri

- İş kullanıcıları tarafından tanımlanır.
- Kod içermez; giriş, çıkış ve şema bilgileri yer alır.
- Orkestrasyonla bir araya getirilen servis kombinasyonlarını temsil eder.
- **Örnek:** Müşteri kaydı, sipariş işleme



Kurumsal (Enterprise) Servisler

- Yeniden kullanılabilir, atomik uygulama servisleridir.
- Domain odaklı geliştirici ekipler tarafından yazılır.
- İş servislerinin yapı taşlarını oluşturur.
- **Örnek:** CreateCustomer, CheckInventory



Uygulama (Application) Servisleri

- Belirli bir uygulamaya özel servislerdir.
- Genellikle yeniden kullanılmaz, uygulamaya özgü çözümler sunar.
- Tek bir ekip tarafından sahiplenilir ve kullanılır.
- **Örnek:** Kampanya hesaplama servisi



Altyapı (Infrastructure) Servisleri

- Teknik ve operasyonel destek sağlar.
- Kurum genelinde kullanılır.
- **Örnek:** Loglama, kimlik doğrulama, yetkilendirme, izleme



Orkestrasyon Motoru

- Dağıtık sistemin kalbidir.
- Servisleri sıralı ve koşullu olarak bir araya getirir.
- Mesaj dönüşümü, hata yönetimi ve koordinasyon sağlar.
- **Not:** Servis başına veritabanı yerine genellikle ortak veritabanı kullanılır.



OD-SOA'nın Gücü

- Servisler arasında **esnek orkestrasyon** sağlar.
- **Yeniden kullanılabilirlik** ve **bakım kolaylığı** sunar.
- İş süreçleri servis tanımı ve orkestrasyonla modellenir.
- Operasyonel işler altyapı servisleriyle ayrıştırılır.

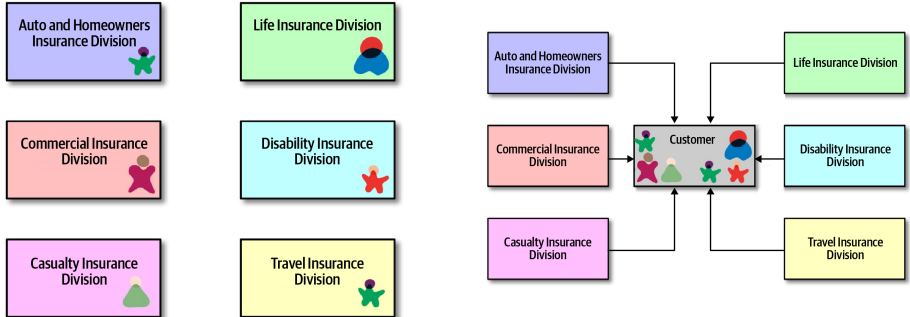


OD-SOA'nın Hedefi: Servis Düzeyinde Yeniden Kullanılabilirlik

- Servis odaklı mimarinin temel hedeflerinden biri: **servis düzeyinde tekrar kullanılabilirlik**.
- Ortak iş davranışları (ör. müşteri yönetimi) **tek bir servis** altında toplanır.
- Bu servis, farklı uygulamalar veya bölümler tarafından tekrar kullanılır.
- **Örnek:** Tüm sigorta bölümleri kendi müşteri tanımını değil, ortak Customer Service bileşenini kullanır.



OD-SOA'nın Hedefi: Servis Düzeyinde Yeniden Kullanılabilirlik



Mesaj Akışının Merkezi: Orkestrasyon Motoru

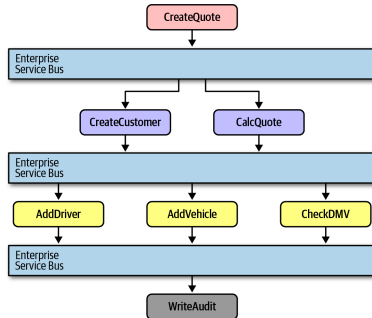
- OD-SOA mimarisinde tüm istekler **orkestrasyon motoru** üzerinden geçer.
- Servisler arası iletişim bile doğrudan yapılmaz; her çağrı önce orkestrasyon katmanına iletilir.
- Bu yapı sayesinde iş mantığı merkezi bir noktadan yönetilir.



Örnek Mesaj Akışı

Senaryo: Kullanıcı teklif almak istiyor.

- 1 CreateQuote iş servisi çağrılır.
- 2 Orkestrasyon motoru, iş akışını başlatır.
- 3 Sırasıyla CreateCustomer ve CalculateQuote servisleri çağrılır.
- 4 Her servis kendi uygulama servislerine başvurur.
- 5 Sonuç, orkestrasyon motoru üzerinden istemciye döner.



Servis Bus'un Rolü

- **Service Bus**, entegrasyon aracı olmasının yanı sıra iş akışlarının tanımlandığı yerdir.
- Akış mantığı (workflow), mesaj yönlendirme, dönüşüm ve hata yönetimi burada gerçekleştirilir.
- Tüm iç ve dış isteklerin merkezi geçiş noktasıdır.



Artılar ve Eksiler

Artılar

- İş süreçleri merkezi yönetilir.
- Tekrarlanabilir ve görsel olarak tanımlanabilir akışlar.
- Gelişmiş loglama, hata yönetimi vb. olanaklar.

Eksiler

- Orkestrasyon motoru tıkanma noktası olabilir.
- Performans gecikmesi yaratabilir.
- İç çağrılar bile karmaşık hale gelir.



Sorun 1: Aşırı Bağımlılık (Tight Coupling)

- Tek bir ortak servis kullanıldığında, tüm diğer servisler bu servise **bağımlı** hale gelir.
- Küçük bir değişiklik bile sistem genelinde etkiler yaratır (*ripple effect*).
- **Sonuç:**
 - Koordine deployment zorunluluğu
 - Geniş kapsamlı test ihtiyacı
 - Mühendislik verimliliğinde düşüş



Sorun 2: Gereksiz Karmaşıklık

- Ortak servis, her ihtiyacı karşılamak zorunda kalır → **şişmiş servis yapıları.**
- **Örnek:** Otomobil sigortası için gereken sürücü belgesi bilgisi, maluliyet sigortası için gereksizdir.
- **Sonuç:**
 - Tüm takımlar aynı karmaşık veri modelini taşımak zorunda
 - Kullanılmayan alanlar, bakım yükü oluşturur



Genel Skor

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★ ★ ★
Evolutionary	★
Fault tolerance	★ ★ ★
Modularity	★ ★ ★
Overall cost	★
Performance	★ ★
Reliability	★ ★
Scalability	★ ★ ★ ★
Simplicity	★
Testability	★



Mikroservis Mimarisi: Giriş

- Mimari stiller genellikle sonradan isimlendirilir—yaygın örüntülerden doğar.
- Mikroservis terimi ise erken bir aşamada ortaya konmuştur.
- Martin Fowler ve James Lewis’in 2014 tarihli blog yazısı: “*Microservices*”
- `https://martinfowler.com/articles/microservices.html`
- Bu yazı, mimari stili tanımlayıp yaygınlaştırmıştır.



Mikroservislerin Kökeni: DDD

- Mikroservisler, **Domain-Driven Design (DDD)** yaklaşımından esinlenmiştir.
- Özellikle **bounded context (sınırlandırılmış bağlam)** kavramı büyük etki yapmıştır.
- Her bounded context, kendi içinde tutarlıdır: kod, veri şeması, davranışlar.
- Dış dünyaya bağımlılık yoktur; başka bağlamlardan kod veya veri paylaşılmaz.



Bağlam Sınırı ve Bağımsızlık

- Geleneksel mimarilerde tekrar kullanılabilirlik amaçlanır → sınıflar ve veritabanları paylaşılır.
- Bu da **yüksek bağlanırlılık (coupling)** getirir.
- Mikroservisler ise yüksek bağımsızlık hedefler:
 - Her servis kendi veritabanına sahiptir.
 - Gerekirse tekrar eden yapı ve veri tanımları barındırılır.
- **Tasarım tercihi: tekrar kullanıma karşı ayırık bağımsızlık.**



Yazılım Mimarisi: İlk Kural

İlk Kural: Her mimari kararda bir ödün vardır.

- **Yeniden kullanım** faydalıdır ama **bağlılık** yaratır.
- Mikroservislerde, **bağlılığı azaltmak** için gerekirse tekrar edilir.
- Özetle: *“Tekrara izin ver ama sınırları net çiz.”*



Duplication over Reuse

- Mikroservislerde bağımsızlık önceliklidir → her servis kendi **Customer** modelini tanımlar.
- Ortak veri modeli yerine, bağlamlara özel (bounded context) modeller kullanılır.
- Bu sayede:
 - Her servis bağımsız geliştirilebilir ve dağıtılabilir.
 - Bir servisteki değişiklik diğerlerini etkilemez.

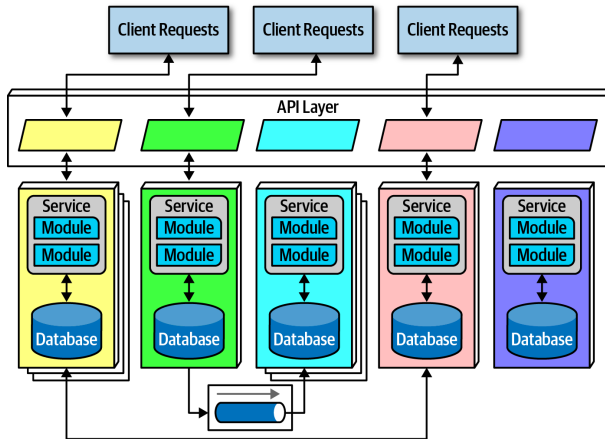
Örnek:

- BillingService yalnızca vergi numarası ile ilgilenir → `Customer.taxId`.
- SupportService yalnızca iletişim tercihi ile ilgilenir → `Customer.contactPreference`.

Aynı kavramın farklı bağlamlarda farklı detaylara ihtiyacı vardır.



Topoloji



Mikroservislerde Dağıtık Yapı

- Her mikroservis kendi sürecinde (process) çalışır: **bağımsız ve izole**.
- Bu yapı önce fiziksel makineleri, sonra sanal makineleri kullanarak yaygınlaştı.
- Paylaşımlı altyapıların sorunları çözülür:
 - Bellek, ağ, disk paylaşımındaki sınırlamalar aşılr.
 - Uygulamalar arası izolasyon sağlanır.
- **Modern bulut ve konteyner teknolojileri** bu yaklaşımı mümkün ve verimli kılar.

Dezavantaj: Dağıtıklık, performans ve güvenlik maliyetleri doğurur.

- Ağ gecikmeleri, endpoint güvenliği, işlem sınırları gibi faktörler dikkatle ele alınmalıdır.
- Servislerin granülerliği başarı için kritiktir.



Granülerlik: Ne Kadar Küçük Olmalı?

- “**Microservice**” kelimesi bir *etikettir*, mutlak bir boyut tanımı değildir.
- Fazla küçük servisler:
 - Sürekli birbirleriyle konuşmak zorunda kalır.
 - İletişim yükü ve karmaşıklığı artar.
- Amaç: **anlamli işlevsel birimler** oluşturmak.

Uygun Granülerlik İçin Kriterler:

- **Amaç:** Her servis, bir işlevsel davranışı tam olarak karşılamalı.
- **Koreografi:** Sürekli haberleşen küçük servisler → birleştirilerek optimize edilebilir.
- **İterasyon:** Doğru granülerlik, deneme-yanılma ile zamanla bulunur.



Mikroservislerde API Katmanı

Görevleri:

- Mikroservislerin dış dünya ile iletişimini sağlar
- HTTP, REST, gRPC vb. protokollerle erişim sunar
- Yalnızca geçiş katmanıdır, iş mantığı içermez

Yapmaması Gerekenler:

- Orkestrasyon yapmamalıdır
- Servisler arası akışı yönetmemelidir (mediatör rolü üstlenmemelidir)



Operational Reuse (İşlemsel Tekrar Kullanım)

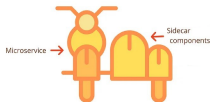
Amaç: Ortak operasyonel işlevlerin tekrar kullanılabilir hale getirilmesi (loglama, izleme, servis keşfi vb.)

Geleneksel SOA:

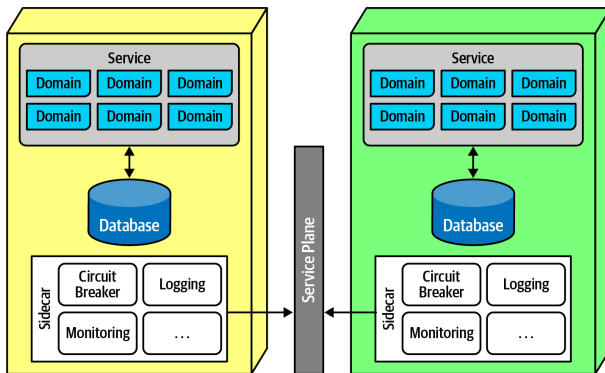
- Hem iş hem de operasyonel servisler birlikte paylaşıldı

Mikroservislerde Çözüm:

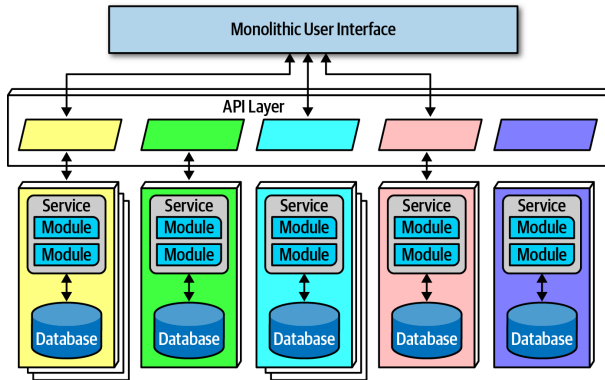
- **Sidecar Pattern:** Her servisin yanına ortak görevleri üstlenen küçük bir yardımcı servis
- **Service Mesh:** Sidecar'ların oluşturduğu ağ üzerinden merkezi yönetim
- **Faydası:** Operasyonel görevlerin standartlaşması, geliştirici bağımsızlığı



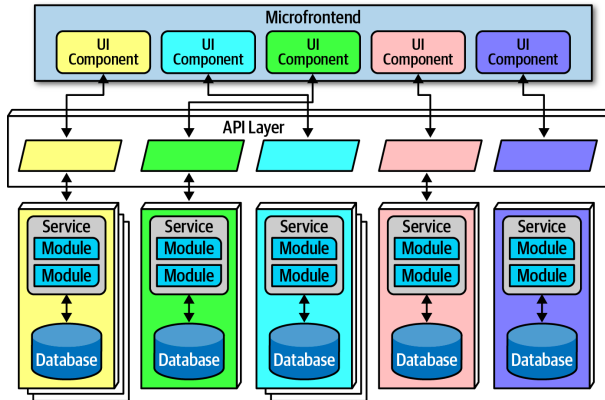
Operational Reuse (İşlemsel Tekrar Kullanım)



Kullanıcı Arayüzü Tasarımı



Kullanıcı Arayüzü Tasarımı



Genel Skor

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★★★★
Evolutionary	★★★★★
Fault tolerance	★★★★
Modularity	★★★★★
Overall cost	★
Performance	★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★★★★

